

SPMP

Presentan

Andres Henao Niño
Joan Emmanuel Orduz Chia
Joshua Prieto Zambrano
David Estevan Rodriguez Jurado

Director
Jaime Andrés Pavlich Mariscal

Pontificia Universidad Javeriana
Facultad de Ingeniería
Trabajo de Grado

Bogotá D.C.
mayo de 2026

Prefacio

El presente documento es el Plan de Gestión del Proyecto (SPMP) del Kit Pagos Colombia, un framework de software construido como trabajo de grado en Ingeniería de Sistemas de la Pontificia Universidad Javeriana. El proyecto es desarrollado por cuatro estudiantes bajo la dirección del Ing. Jaime Andrés Pavlich Mariscal, con el propósito de producir un artefacto que simplifique la integración de las principales pasarelas de pago colombianas en aplicaciones de software.

El framework está compuesto por tres componentes: un SDK con interfaz unificada para Wompi, PayU, Mercadopago y Kushki; una API de simulación desplegada que permite realizar pruebas sin depender de credenciales externas; y una documentación centralizada de datos de prueba adaptados al contexto colombiano. El impacto esperado es reducir la complejidad y el tiempo que los equipos de desarrollo invierten al integrar múltiples proveedores de pago, concentrando en un único punto de entrada la lógica que hoy está distribuida entre cuatro fuentes distintas.

Este documento describe cómo se va a ejecutar el proyecto: las decisiones metodológicas que lo estructuran, la organización del equipo, el plan de trabajo, los mecanismos de seguimiento y los procesos de calidad que guiarán cada fase. No describe el producto en detalle técnico; para eso está el documento de propuesta (VFP).

La audiencia principal de este SPMP es el equipo de desarrollo, que lo utiliza como referencia operativa. También es relevante para el director del trabajo de grado en su rol de supervisión y aprobación de entregables, y para los evaluadores académicos que revisen el proceso de ejecución. Puede ser útil, adicionalmente, para otros estudiantes que desarrollen proyectos similares bajo Design Science Research.

Tabla de Contenidos

Historial de Cambios 2
Prefacio 3
Tabla de Contenidos 4
Lista de Figuras 6
Lista de Tablas 7
1. Vista General del Proyecto 8
1.1. Visión del Producto 9
1.2. Propósito, Alcance y Objetivos 9
1.2.1. Propósito 9
1.2.2. Alcance 10
1.2.3. Objetivos 10
1.3. Supuestos y Restricciones 11
1.3.1. Supuestos 11
1.3.2. Restricciones 11
1.4. Entregables 11
1.5 Evolución del Plan 13
2. Glosario 1
3. Contexto del proyecto 2
3.1. Modelo de Ciclo de Vida 2
3.1.2. Adaptación del ciclo de vida 2
3.1.3 Hitos del ciclo de vida 3
3.1.4. Análisis de Alternativas y Justificación 4
3.2. Lenguajes y Herramientas 6
3.2.1 Lenguajes de modelado 6
3.2.2. Lenguajes de programación 7
3.2.3. Herramientas de desarrollo y arquitectura 7
3.2.4. Herramientas de análisis, modelado y documentación 8
3.2.5. Herramientas de gestión del proyecto y comunicación 8
3.2.6. Herramientas de ofimática y presentación 9
3.2.7. Análisis de Alternativas y Justificación 9
3.3. Plan de Aceptación del Producto 10
3.3.1. Prototipo funcional del SDK 10
3.3.2. API de simulación desplegada 10
3.3.3. Documentación centralizada de datos de prueba 11
3.3.4. Informe comparativo de métricas CK 11
3.3.5. Documentación final del trabajo de grado y repositorio público 11
3.4. Organización del Proyecto y Comunicación 11
3.4.1. Interfaces Externas 11
3.4.2. Organigrama y Descripción de Roles 13

4. Administración del Proyecto....14
4.1. Métodos y Herramientas de Estimación....14
4.1.1. Estimación del tamaño del software....14
4.2. Inicio del proyecto....16
4.2.1. Entrenamiento del Personal....16
4.2.2. Infraestructura....18
4.3. Planes de Trabajo del Proyecto....20
4.3.1. Descomposición de Actividades....20
4.3.2. Calendarización....22
4.3.3. Asignación de Recursos....23
4.3.4. Asignación de Presupuesto y Justificación....24
5. Monitoreo y Control del Proyecto....26
5.1. Administración de Requerimientos....26
5.1.1. Artefactos del proceso....26
5.1.2. Proceso de administración de requerimientos....26
5.2. Monitoreo y Control de Progreso....27
5.2.1. Unidades de medición del progreso....27
5.2.2. Actividades de reporte de progreso....28
5.2.3. Acciones correctivas....28
5.3. Cierre del Proyecto....30
5.3.1. Cierre de fase....31
5.3.2. Cierre final del proyecto....31
6. Entrega del Producto....32
7. Procesos de Soporte....34
7.1. Ambiente de Trabajo....34
7.2. Análisis y Administración de Riesgos....35
7.3. Administración de Configuración y Documentación....36
7.4. Métricas y Proceso de Medición....40
7.5. Control de Calidad....42
8. Referencias....45

Lista de Figuras

Figura 1. Adaptación del Ciclo de Vida..... 15

Lista de Tablas

Tabla 1. Entregables.....12
Tabla 2. Interfaces Externas.....20
Tabla 3. Descripción de los roles.....21

1. Vista General del Proyecto

El proyecto Kit Pagos Colombia surge a partir de un problema recurrente en el desarrollo de software colombiano: integrar pasarelas de pago sigue siendo un proceso fragmentado, repetitivo y, en muchos casos, innecesariamente complejo. Plataformas como Wompi, PayU, Mercado Pago y Kushki ofrecen servicios similares, pero cada una maneja estructuras distintas para autenticación, respuestas HTTP, validaciones y notificaciones asíncronas. En la práctica, esto obliga a los equipos de desarrollo a implementar varias veces la misma lógica, aumentando el tiempo de integración y la probabilidad de errores.

De acuerdo con la Cámara Colombiana de Comercio Electrónico, durante 2025 las transacciones digitales en Colombia superaron los 140 billones de pesos ("Informe de comercio minorista / Diciembre 2025", s. f.), lo que evidencia el crecimiento sostenido del comercio electrónico y la necesidad de herramientas que simplifiquen la incorporación de servicios de pago dentro de aplicaciones modernas. A pesar de esto, el ecosistema colombiano continúa dependiendo de APIs heterogéneas que exigen procesos de integración independientes para cada proveedor.

Frente a este escenario, el proyecto propone la construcción de un framework open source que permita unificar la interacción con distintas pasarelas bajo una misma interfaz de programación. La intención no es reemplazar las APIs oficiales de los proveedores, sino construir una capa intermedia que reduzca la complejidad técnica que enfrentan los desarrolladores cuando necesitan trabajar con múltiples pasarelas dentro de una misma aplicación.

El framework está compuesto por tres elementos principales. El primero es un SDK con interfaz unificada que abstrae las diferencias entre Wompi, PayU, Mercado Pago y Kushki para operaciones comunes como creación de pagos, consulta de estados y recepción de webhooks. El segundo componente es una API de simulación desplegada, diseñada para facilitar pruebas de integración sin depender completamente de los entornos sandbox externos. Finalmente, el proyecto incluye una documentación centralizada con datos de prueba y configuraciones utilizadas por cada proveedor, buscando reducir el tiempo que normalmente se pierde navegando entre distintas documentaciones oficiales.

El propósito principal del proyecto es desarrollar y evaluar una solución que simplifique la integración de múltiples pasarelas de pago en el contexto colombiano. Más allá de construir software funcional, el trabajo busca analizar si una arquitectura unificada realmente reduce la complejidad de implementación frente a una integración tradicional realizada directamente sobre las APIs nativas de cada proveedor.

La propuesta se desarrolla bajo el enfoque de Design Science Research (DSR), metodología orientada a proyectos cuyo resultado principal es la construcción y validación de un artefacto tecnológico (Peffers, Tuunanen, Rothenberger & Chatterjee, 2007). Esto implica que el framework no solamente debe funcionar correctamente, sino que además debe evaluarse mediante criterios técnicos verificables. Para ello, el proyecto compara integraciones realizadas con el framework frente a integraciones tradicionales utilizando las métricas WMC, CBO y RFC del conjunto de métricas CK propuesto por Chidamber y Kemerer (1994).

El diseño del framework se apoya principalmente en la arquitectura hexagonal propuesta por Cockburn (2024), la cual permite separar el núcleo de negocio de las dependencias externas mediante puertos y adaptadores. Esta decisión arquitectónica resulta especialmente útil en un

ecosistema como el de las pasarelas de pago, donde los proveedores cambian constantemente sus mecanismos internos y sus convenciones de integración. Adicionalmente, se emplean patrones de diseño del catálogo GoF cómo Strategy, Facade y Factory para encapsular el comportamiento específico de cada pasarela y ofrecer una interfaz homogénea hacia el desarrollador (Gamma, Helm, Johnson & Vlissides, 1994).

El alcance del proyecto está delimitado a cuatro pasarelas utilizadas activamente en Colombia: Wompi, PayU, Mercado Pago y Kushki. El framework cubre operaciones esenciales relacionadas con pagos digitales y manejo de notificaciones asíncronas, además de la simulación de escenarios controlados de prueba. Sin embargo, no contempla procesamiento financiero real, integración con sistemas contables, interfaces gráficas de pago ni soporte para proveedores distintos a los definidos inicialmente. Tampoco modifica los mecanismos internos de seguridad implementados por las pasarelas, ya que el framework actúa únicamente como intermediario entre el cliente y los proveedores externos.

Como resultado del proyecto se espera obtener un SDK funcional basado en arquitectura hexagonal, una API REST de simulación desplegada, documentación pública de datos de prueba, un informe comparativo de métricas de evaluación y el repositorio público del framework con documentación técnica de uso. La intención es que estos entregables no se limiten únicamente al contexto académico del trabajo de grado, sino que puedan servir como base reutilizable para desarrolladores y futuros proyectos relacionados con integración de pagos en Colombia.

1.1. Visión del Producto

El ecosistema colombiano de pasarelas de pago está fragmentado. Wompi, PayU, Mercado Pago y Kushki funcionan cada una con sus propias convenciones de autenticación, estructuras de respuesta distintas y mecanismos de notificación asíncrona particulares. Para un equipo de desarrollo que necesita integrar más de una, esa fragmentación se convierte en trabajo repetitivo: hay que estudiar cuatro documentaciones, implementar cuatro lógicas de autenticación y normalizar manualmente los formatos de respuesta. El tiempo invertido crece con cada proveedor adicional y la probabilidad de errores también.

El Kit Pagos Colombia busca resolver eso. Es un framework open source que unifica la integración de estas pasarelas bajo una interfaz común, de modo que un desarrollador pueda interactuar con varias de ellas usando la misma lógica de código. Además, incluye una API de simulación que elimina la dependencia de los entornos sandbox externos durante el desarrollo, y una documentación que centraliza los datos de prueba dispersos en múltiples portales.

La visión de largo plazo es que el kit funcione como referencia técnica dentro del ecosistema fintech colombiano, reduciendo las barreras de entrada para que proyectos de menor escala puedan incorporar múltiples opciones de pago sin duplicar esfuerzo. Este trabajo de grado construye y evalúa la primera versión funcional de esa visión.

1.2. Propósito, Alcance y Objetivos

1.2.1. Propósito

Este proyecto se realiza como trabajo de grado en Ingeniería de Sistemas. Su propósito es construir y evaluar un artefacto de software que resuelva el problema de la fragmentación en el proceso

de integración de pasarelas de pago en Colombia. La construcción del artefacto se enmarca en Design Science Research, que exige no solo que el sistema sea desarrollado correctamente, sino que sea validado en escenarios representativos del problema original y que sus resultados sean comunicados de forma rigurosa.

1.2.2. Alcance

El proyecto incluye el diseño, desarrollo y evaluación de tres componentes:

- El SDK con interfaz unificada: una biblioteca de software que abstrae las diferencias entre las APIs de Wompi, PayU, Mercado Pago y Kushki para las operaciones de creación de pagos, consulta de estado y gestión de notificaciones asíncronas (webhooks).
- La API de simulación: un servidor REST desplegado que replica el comportamiento de las cuatro pasarelas en cuatro escenarios de prueba controlados (pago exitoso, pago rechazado, error de red y notificación asíncrona).
- La documentación centralizada de datos de prueba: consolidación en un repositorio público de los datos de tarjetas, identificaciones y referencias de pago que cada pasarela provee en su entorno sandbox.

El proyecto no incluye: procesamiento real de transacciones financieras; interfaces de usuario para flujos de pago; cobertura de pasarelas distintas a las cuatro mencionadas; integración con sistemas contables o ERP; ni responsabilidad sobre la continuidad de las APIs externas de los proveedores. La capa de seguridad propia de cada pasarela (autenticación con tokens, cifrado de datos, cumplimiento PCI-DSS) tampoco es modificada; el framework delega correctamente hacia cada proveedor sin interferir en sus mecanismos internos.

1.2.3. Objetivos

El objetivo general es desarrollar el Kit Pagos Colombia para simplificar la integración de múltiples pasarelas de pago en el contexto colombiano.

Los objetivos específicos son seis.

- **Analizar** las características técnicas, mecanismos de integración y consideraciones de seguridad de las cuatro pasarelas seleccionadas.
- **Diseñar** la arquitectura del framework aplicando la arquitectura hexagonal y patrones GoF.
- **Desarrollar** un prototipo funcional del framework que implemente la interfaz unificada y los mecanismos de comunicación segura.
- **Implementar** la API de simulación.
- **Consolidar** los datos de prueba por pasarela para su documentación.
- **Evaluar** el framework mediante su aplicación en proyectos prototípicos usando las métricas WMC, CBO y RFC de Chidamber y Kemerer (1994), comparando los resultados frente a una integración directa con las APIs nativas.

1.3. Supuestos y Restricciones

1.3.1. Supuestos

El proyecto asume que las APIs de Wompi, PayU, Mercado Pago y Kushki se mantendrán estables en sus versiones actuales durante el período de desarrollo. Un cambio incompatible hacia atrás en alguna de ellas obligaría a rediseñar el adaptador correspondiente, lo que impactaría el cronograma. Esta suposición es razonable dado que las APIs de pasarelas de pago suelen tener versiones con ciclos de vida largos por las implicaciones que tienen sobre sistemas productivos.

Se asume que los entornos sandbox de las cuatro pasarelas estarán disponibles para consulta y prueba durante todas las fases del proyecto. La indisponibilidad prolongada de alguno afectaría la implementación y verificación del adaptador correspondiente.

Se asume que los cuatro integrantes del equipo dedicarán el tiempo acordado al proyecto durante el semestre, y que el director mantendrá disponibilidad para las reuniones de seguimiento establecidas. También se asume que es técnicamente viable construir una capa de abstracción suficientemente homogénea sobre las cuatro pasarelas para las operaciones fundamentales; esto está respaldado por trabajos previos como el de Manuaba e Hisyam (2022), aunque su validez específica para el conjunto colombiano se verificará en la fase de análisis.

1.3.2. Restricciones

El proyecto está acotado a un semestre académico, lo que limita la evaluación a proyectos prototípicos construidos por el propio equipo. El framework se distribuirá bajo la licencia Apache 2.0 , la cual permite usar, modificar y distribuir software libremente, incluyendo fines comerciales y creaciones propietarias, sin obligación de liberar el código fuente de las modificaciones, siempre que se incluyan los avisos de derechos de autor y cambios (Apache Software Foundation, 2004). El despliegue de la API de simulación se realizará en una plataforma de nube con plan gratuito o de bajo costo (como Railway o Render), lo que impone restricciones de disponibilidad y rendimiento que deben considerarse durante las pruebas. La elección del lenguaje de programación del SDK impone restricciones sobre las herramientas disponibles para calcular las métricas CK, aspecto que se aborda en la sección de Lenguajes y Herramientas.

1.4. Entregables

Entregable	**Estándares asociados**	**Justificación**
Prototipo funcional del SDK con interfaz unificada para Wompi, PayU, Mercadopago y Kushki	Cockburn, A. (2024). *Hexagonal Architecture Explained*; Gamma, E., Helm, R., Johnson, R., & Vlissides, J. *Design Patterns: Elements*	La arquitectura hexagonal establece el estilo estructural del SDK, garantizando que el núcleo de lógica de pagos sea independiente de los proveedores. Los patrones GoF (Strategy, Facade y Factory) fundamentan las decisiones de diseño de la interfaz unificada y los adaptadores. Los patrones de integración empresarial de Hohpe y Woolf justifican el

	of Reusable Object-Oriented Software; Hohpe, G. & Woolf, B. (2003). *Enterprise Integration Patterns*	mecanismo de normalización de respuestas y el manejo de comunicaciones asíncronas entre el framework y las pasarelas.
API de simulación desplegada con endpoints que replican el comportamiento de las pasarelas	Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*; Ammann, P., & Offutt, J. (2016). *Introduction to Software Testing*	Los principios REST de Fielding fundamentan el diseño de la interfaz de la API de simulación, garantizando una interfaz coherente, sin estado y desacoplada de cualquier cliente. Los criterios de Ammann y Offutt justifican la cobertura de escenarios que debe exponer la API (éxito, rechazo, error de red y notificaciones asíncronas) y el entorno controlado equivalente a un sandbox.
Documentación centralizada de datos de prueba por pasarela	Ammann, P., & Offutt, J. (2016). *Introduction to Software Testing*	La noción de entorno sandbox como espacio de verificación aislado, definida por Ammann y Offutt, justifica la necesidad de consolidar los datos de prueba de cada proveedor en un formato accesible y estructurado, reduciendo la dependencia de fuentes dispersas durante el desarrollo.
Informe comparativo de métricas de evaluación (WMC, CBO, RFC)	Chidamber, S. R. & Kemerer, C. F. (1994). *A metrics suite for object oriented design*. IEEE Transactions on Software Engineering, 20(6), 476–493	Las métricas CK (Chidamber y Kemerer) proveen una herramienta cuantitativa y reproducible para comparar la complejidad, el acoplamiento y la superficie de interacción del código de integración con y sin el framework. Su uso garantiza que la evaluación del artefacto no dependa exclusivamente de observaciones subjetivas.
Documento final del trabajo de grado y repositorio público con documentación de uso	Peffers, K., Tuunanen, T., Rothenberger, M., & Chatterjee, S. (2007). *A design science research methodology for*	DSR exige que los artefactos producidos y las lecciones aprendidas sean comunicados de forma rigurosa a las audiencias relevantes. El documento de grado y el repositorio público materializan esta exigencia, presentando el proceso de construcción, los resultados de la

	information systems research. Journal of Management Information Systems, 24, 45–77	evaluación y las condiciones bajo las cuales el framework resulta útil.

Tabla 1. Entregables.

1.5 Evolución del Plan

Este documento no es estático. A medida que el proyecto avance y se produzcan ajustes en el alcance, el cronograma, los riesgos o las decisiones técnicas, el SPMP deberá actualizarse para reflejar el estado real del proyecto. Cada entrega parcial del trabajo de grado es una oportunidad obligatoria de revisión, especialmente de la carta Gantt descrita en la sección de **Calendarización**, que debe estar actualizada en cada entrega.

Cualquier integrante puede proponer un cambio al plan. La propuesta se discute en la reunión semanal del equipo y, si afecta el alcance o los criterios de evaluación definidos con el director, se comunica antes de implementarse. Los cambios aprobados se registran en el historial de cambios del documento con fecha, autor y descripción breve. El SPMP se versiona en el repositorio del proyecto bajo control de cambios en GitHub.

Los cambios que no requieren aprobación del director son: ajustes al cronograma que no afecten los hitos principales, actualización de la tabla de riesgos, correcciones menores de redacción y actualización de métricas de progreso. Los que sí requieren alineación son:

- Modificaciones al alcance del proyecto
- Cambios en los entregables o sus criterios de aceptación
- Cambios en la metodología de evaluación.

Se anticipa al menos una actualización por cada hito completado, con una revisión mayor al finalizar la fase de desarrollo para reflejar los resultados de las pruebas internas y los ajustes que hayan surgido durante la implementación de los adaptadores.

2. Glosario

A continuación se encuentran los conceptos y acrónimos relevantes para el entendimiento del Kit Pagos Colombia:

- **Pasarela de pago:** Una pasarela de pago es un sistema de software que actúa como intermediario entre un comercio y las entidades financieras, autorizando y verificando transacciones de pago en línea mediante la transmisión segura de datos entre el comprador, el comercio y el banco procesador. (Hisyam & Manuaba, 2022)
- **API - Interfaz de programación de aplicaciones:** Una API es un contrato de comunicación entre sistemas de software que define cómo un programa puede solicitar servicios o datos a otro, sin necesidad de conocer su funcionamiento interno. En el contexto de pagos digitales, las APIs permiten que una aplicación comercial interactúe con una pasarela de pago para crear transacciones, consultar estados y recibir resultados de forma estandarizada.(Fielding, 2000)
- **Framework::** En ingeniería de software, un framework se entiende como una estructura compuesta por componentes reutilizables que permiten estandarizar y agilizar el desarrollo de aplicaciones, apoyándose en principios de reutilización y en el uso de estándares definidos (Amazon Web Services, s. f.).
- **Patrones de diseño:** Los patrones de diseño son soluciones recurrentes y documentadas a problemas comunes en el diseño de software. No son código reutilizable directamente, sino descripciones o plantillas que guían cómo estructurar componentes para resolver un problema específico en un contexto determinado. (*Patrones de Diseño GoF Por Pagina | PDF*, s. f.).
- **GoF:** En ingeniería de software, GoF es un acrónimo de "Gang of Four"(Pandilla de cuatro) que hace referencia a los cuatro autores (Erich Gamma, Richard Helm, Ralph Johnson y John Vlisside) del libro de 1994, *Design Patterns: Elements of Reusable Object-Oriented Software.* Definen 23 patrones de diseño óptimos para resolver problemas reiterados en la programación orientada a objetos. (*Patrones de Diseño GoF Por Pagina | PDF*, s. f.).
- **Arquitectura hexagonal:** La arquitectura hexagonal, propuesta por Alistair Cockburn, es un estilo que organiza una aplicación alrededor de un núcleo de lógica de negocio aislado. Este núcleo se comunica mediante puertos y adaptadores que lo conectan con tecnologías externas como bases de datos, APIs o interfaces de usuario. Su objetivo es mantener el núcleo independiente de cualquier tecnología o proveedor externo. (*Hexagonal architecture pattern - AWS Prescriptive Guidance*, s. f.)
- **Normalización de APIs:** La normalización de APIs es el proceso de unificar las respuestas, estructuras de datos y comportamientos de múltiples servicios externos bajo un formato común y consistente. En sistemas que integran proveedores heterogéneos, la normalización reduce el acoplamiento entre el consumidor y cada proveedor individual, facilitando la intercambiabilidad de servicios. (Taherkordi et al., 2011)
- **Entornos de prueba:** Un entorno sandbox en el contexto de APIs de pago es un ambiente de simulación aislado provisto por el proveedor que replica el comportamiento de su sistema productivo sin procesar transacciones reales. Permite a los desarrolladores verificar la correcta integración de sus sistemas durante la fase de desarrollo, reduciendo el riesgo de errores en producción. (Ammann & Offutt, 2016)
- **Integración de sistemas:** La integración de sistemas es el proceso de conectar subsistemas o componentes independientes para que funcionen como un conjunto coordinado. En arquitecturas orientadas a servicios, la integración implica gestionar diferencias en protocolos, formatos de datos y modelos de comunicación entre sistemas que no fueron diseñados para interoperar directamente.(*Hohpe, G. and Woolf, B. (2003) Enterprise Integration Patterns Designing,*

Building, and Deploying Messaging Solutions. Addison-Wesley. - References - Scientific Research Publishing, s. f.)

- **Webhook**: Un webhook es un mecanismo de notificación asíncrona entre sistemas mediante el cual un proveedor envía una solicitud HTTP POST hacia una URL registrada por el receptor cuando ocurre un evento determinado. En el contexto de pasarelas de pago, los webhooks notifica el resultado final de una transacción una vez que el procesamiento bancario ha concluido, de forma independiente a la sesión HTTP original. (Hohpe & Woolf, 2003).
- **SDK (Software Development Kit):** Un SDK es un conjunto de herramientas, librerías, documentación y ejemplos de código que permiten a los desarrolladores construir aplicaciones que interactúen con una plataforma o servicio específico. En el contexto de este proyecto, el SDK del Kit Pagos Colombia es la biblioteca principal que abstrae la lógica de integración con las pasarelas de pago. (Amazon Web Services, s. f.).
- **Métricas CK (Chidamber y Kemerer):** Conjunto de métricas orientadas a objetos propuesto por Chidamber y Kemerer (1994) para medir la complejidad, el acoplamiento y otras propiedades estructurales del código. Las tres métricas usadas en este proyecto son WMC (Weighted Methods per Class), CBO (Coupling Between Object Classes) y RFC (Response For a Class). (Chidamber & Kemerer, 1994)
- **DSR (Design Science Research):** Metodología de investigación utilizada en sistemas de información e ingeniería de software cuando el resultado central de la investigación es la construcción y evaluación de un artefacto que resuelve un problema real. Organiza el trabajo en seis fases: identificación del problema, definición de objetivos, diseño, desarrollo, demostración y comunicación. (Peffers et al., 2007).

3. Contexto del proyecto

3.1. Modelo de Ciclo de Vida

La naturaleza del proyecto determina cuál metodología se elige antes de analizar las alternativas. Cuando el aporte de un trabajo investigativo no es una teoría ni la observación de algún fenómeno, sino la construcción de un sistema que resuelve un problema real, la metodología más acorde al Kit Pagos Colombia es Design Science Research (DSR) (Peffers, K., Tuunanen, T., Rothenberger, M., & Chatterjee, S, 2007). Esta metodología organiza el proceso alrededor de una pregunta central: ¿el artefacto fue construido correctamente y resuelve el problema para el que fue diseñado? Esas dos dimensiones, estructuran las fases del proyecto y le dan coherencia a la investigación.

3.1.2. Adaptación del ciclo de vida

- **Identificación y definición del problema:** Consiste en estudiar las documentaciones técnicas de Wompi, PayU, Mercado Pago y Kushki para identificar sus diferencias en aspectos relevantes para su implementación tales como, autenticación, estructuras de respuesta, manejo de webhooks y comportamiento ante errores. El resultado de esta fase es una especificación técnica que traduce la fragmentación del ecosistema en requerimientos funcionales concretos para el framework. Esta fase produce el mapa del ecosistema y el repositorio de parámetros de prueba.
- **Definición de objetivos:** A partir del análisis anterior se delimita el alcance del framework: qué operaciones debe soportar, bajo qué criterios se seleccionaron las pasarelas, cómo se medirá el éxito del proyecto y cuales son las características que deben tener los proyectos prototípicos usados en la evaluación. Esta fase produce el documento de alcance, los criterios

de evaluación formalizados y el backlog inicial.

- **Diseño:** Aquí las decisiones del documento de alcance se convierten en contratos arquitectónicos verificables: el modelo de dominio unificado del framework, los puertos de la arquitectura hexagonal, la especificación de los adaptadores por cada pasarela y el diseño de la API de simulación con sus escenarios de prueba. Esta fase produce un conjunto de artefactos de diseño que orientan el desarrollo sin ambigüedad y que también funcionan como criterios de corrección durante la implementación.
- **Desarrollo:** Es la fase más extensa del proyecto y la única que opera de forma iterativa internamente. Se construye en tres iteraciones dependientes:
    - El núcleo del framework con sus contratos de puerto.
    - Los adaptadores de cada pasarela con bajo acoplamiento, siendo independientes entre sí.
    - API de simulación junto con la documentación de datos de prueba y las pruebas internas.

      El orden no es arbitrario: la arquitectura hexagonal impone que el núcleo y sus contratos existan antes de que cualquier adaptador pueda construirse correctamente.
- **Demostración y evaluación:** El framework se aplica en los proyectos prototípicos definidos en la fase 2 (definición de objetivos) y sobre el código resultante se calculan las métricas definidas en el documento de la propuesta, WMC, CBO y RFC de Chidamber y Kemerer (1994). Estas métricas se comparan a las de una integración directa con las APIs nativas, lo que constituye el núcleo experimental del proyecto. En paralelo de esta evaluación, sucede la demostración, siendo esta el acto de integrar el framework en los prototipos.
- **Comunicación:** Esta fase comprende la consolidación del documento de trabajo de grado y la publicación del repositorio público con documentación suficiente para que un desarrollador externo pueda integrar el kit sin haber estado involucrado en el desarrollo del mismo. Aunque esta fase cierra formalmente el ciclo DSR, su redacción comienza desde las primeras fases: el documento de grado no es una actividad de cierre sino un artefacto que crece en paralelo con el proyecto.

3.1.3 Hitos del ciclo de vida

Los seis hitos que marcan el avance del ciclo de vida son:

- H1: el mapa del ecosistema completo y el repositorio de parámetros de prueba validado.
- H2: la arquitectura del framework diseñada con contratos de puerto especificados.
- H3: el núcleo del SDK funcional con al menos un adaptador operativo.
- H4: los cuatro adaptadores y la API de simulación completos y con pruebas internas aprobadas; H5, los proyectos prototípicos evaluados con las métricas CK (Chidamber, Kemerer, 1994) registradas; y H6, el repositorio público publicado y el documento de grado entregado.

La Figura 1 ilustra el ciclo de vida del proyecto. La fase 4 al ser el desarrollo se tomará como iterativa de manera interna a la misma con el fin de garantizar un desarrollo ágil.

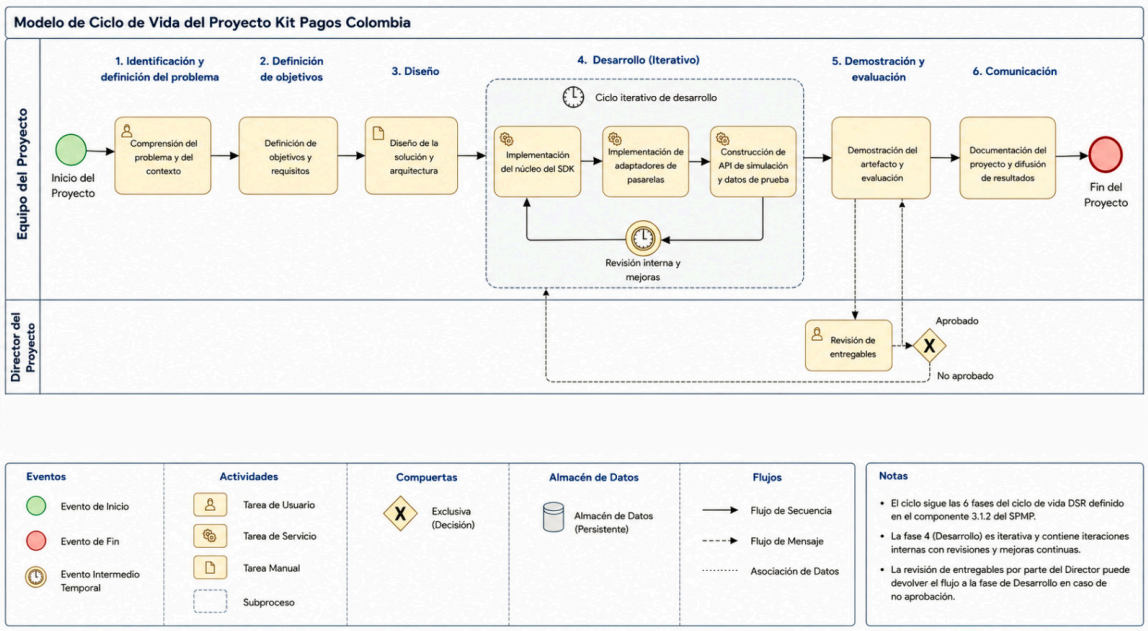


Figura 1. Adaptación del Ciclo de Vida.

3.1.4. Análisis de Alternativas y Justificación

Antes de justificar la elección de DSR, es necesario revisar los modelos que se consideraron y por qué no resultan suficientes para este proyecto.

Modelo en cascada

El modelo en cascada estructura el desarrollo en fases secuenciales: análisis de requisitos, diseño, implementación, pruebas y despliegue, con transiciones unidireccionales entre ellas. Su mayor fortaleza es la claridad estructural que ofrece, lo que facilita la planificación anticipada y la documentación sistemática. Sin embargo, su rigidez resulta problemática para proyectos en los que el entendimiento del problema evoluciona a medida que se trabaja en él.

En el caso del Kit Pagos Colombia, la fase de análisis de las APIs consiste en estudiar cuatro documentaciones técnicas distintas, y es habitual que ese proceso revele detalles que afecten las decisiones de diseño. El modelo en cascada no tiene mecanismos formales para incorporar esos hallazgos una vez que la fase de análisis ha cerrado. Adicionalmente, la dimensión investigativa del proyecto (evaluar el artefacto y compararlo con un enfoque alternativo) no tiene lugar natural dentro del ciclo cascada, que asume que el sistema se entrega y eso constituye el fin del proceso.

Modelo incremental

El modelo incremental mejora la adaptabilidad del cascada al proponer la entrega del sistema en partes funcionales progresivas. Cada incremento produce una versión del sistema que puede ser evaluada antes de proceder al siguiente. Esto es especialmente útil cuando los requisitos no están completamente definidos desde el inicio.

Sin embargo, el modelo incremental no proporciona un marco metodológico para la generación de

conocimiento académico ni para la validación científica de artefactos. En este proyecto, la evaluación no es simplemente "funciona o no funciona": es una comparación cuantitativa entre dos enfoques de integración usando métricas de ingeniería de software, lo que requiere un diseño experimental explícito. El modelo incremental no contempla ese componente. Adicionalmente, la arquitectura hexagonal impone un orden de construcción específico (núcleo antes que adaptadores) que no corresponde necesariamente al orden de valor de negocio que guía un incremento típico.

Scrum

Scrum es el marco de trabajo ágil más extendido. Organiza el desarrollo en sprints cortos con retrospectivas, backlog priorizado y participación activa de un Product Owner que representa las necesidades del usuario final. Sus fortalezas en adaptabilidad y entrega continua de valor son bien conocidas.

El problema de aplicar Scrum como marco principal en este proyecto es doble. Por un lado, no hay un cliente externo: el Product Owner sería en la práctica el equipo mismo, lo que elimina el mecanismo de validación externa que Scrum presupone. Por otro lado, Scrum tampoco ofrece un marco para la contribución académica ni para la evaluación sistemática del artefacto contra un escenario de control. Un proyecto de grado no puede limitarse a entregar software funcional; debe justificar por qué ese software resuelve el problema mejor que las alternativas existentes, y para eso necesita un método.

Esto no significa que Scrum sea irrelevante para el proyecto. Dentro de la fase de desarrollo de DSR, los principios de Scrum son útiles para organizar el trabajo iterativo: el backlog permite priorizar componentes, los sprints cortos (de una semana) ayudan a mantener el ritmo, y las revisiones periódicas facilitan la detección temprana de problemas de implementación. Scrum se incorpora entonces como práctica interna de la fase 4, no como marco metodológico del proyecto completo.

Design Science Research (DSR)

DSR fue propuesto por Peffers et al. (2007) como metodología para proyectos cuyo resultado central es la construcción de un artefacto que resuelve un problema real. A diferencia de los modelos anteriores, DSR integra explícitamente la investigación y la construcción: no basta con que el sistema funcione, debe evaluarse su efectividad frente al problema que motivó su creación y los resultados deben comunicarse de forma rigurosa.

Las seis fases adaptadas de DSR corresponden directamente a lo que este proyecto necesita hacer: identificar y caracterizar el problema de la fragmentación de las APIs colombianas de pago; definir los objetivos del framework y los criterios con que se medirá su éxito; diseñar la arquitectura del SDK y la API de simulación; construir el artefacto; demostrarlo y evaluarlo con las métricas CK; y comunicar los resultados a través del documento de grado y el repositorio público.

Ninguno de los modelos anteriores ofrece ese recorrido completo. DSR es la única metodología que exige los tres elementos que este proyecto requiere: rigor en el diseño del artefacto, un mecanismo de evaluación experimental y un componente de comunicación académica.

Prácticas específicas adoptadas

El modelo de ciclo de vida no se ejecuta de forma monolítica. Para los aspectos más importantes del proyecto se adoptan prácticas específicas que complementan DSR.

Para la fase de análisis de APIs, se utiliza revisión documental estructurada: cada integrante estudia la documentación oficial de una o dos pasarelas y produce un mapa de diferencias técnicas que luego se consolida en equipo. Esto garantiza que el análisis sea sistemático y que los hallazgos queden documentados de forma trazable.

Para las decisiones de diseño arquitectónico, se usan documentos breves que registran qué decisión se tomó, qué alternativas se consideraron y por qué se eligió la opción seleccionada. Esto es especialmente útil para decisiones sobre la estructura de los puertos del SDK y las convenciones de normalización de respuestas, que no siempre son obvias y tienden a revisitarse durante el desarrollo.

Para el desarrollo de los adaptadores, que son el componente de mayor complejidad técnica, se aplica programación en pares. Dos integrantes trabajan juntos en cada adaptador, reduciendo la probabilidad de errores y distribuyendo el conocimiento específico de cada pasarela dentro del equipo. El núcleo del SDK, por su naturaleza más conceptual y por ser la base sobre la que depende todo lo demás, es revisado por todos los integrantes antes de que comience la implementación de los adaptadores.

Para las pruebas, el framework se valida primero contra la API de simulación propia antes de validarse contra los sandboxes reales de las pasarelas. Esto reduce la dependencia de credenciales externas durante el desarrollo y permite pruebas más controladas y reproducibles. Las pruebas unitarias sobre el núcleo y las pruebas de integración sobre los adaptadores se automatizan desde el principio y se ejecutan en cada pull request mediante integración continua.

Para la revisión de código, se adopta una política estricta de pull requests: ningún cambio sobre la rama principal del repositorio se hace sin la revisión de al menos otro integrante. Esto funciona como mecanismo de control de calidad continuo y también como transferencia de conocimiento entre quienes trabajan en frentes distintos del framework.

Para la evaluación con métricas CK, el diseño experimental se establece desde la fase 2 y no se improvisa al llegar a la fase 5. Los proyectos prototípicos, las clases sobre las que se calculan las métricas y la herramienta de medición se definen con anticipación, de modo que la comparación entre integración directa e integración con el framework sea reproducible y no esté sujeta a sesgos de selección posterior.

3.2. Lenguajes y Herramientas

Las herramientas y lenguajes seleccionados para el proyecto responden a dos criterios principales: la coherencia con los objetivos técnicos del framework y la familiaridad del equipo. A continuación se describen por categoría.

3.2.1 Lenguajes de modelado

Herramienta	**Justificación y aplicación en el proyecto**
UML	Es el lenguaje de modelado principal del proyecto. Se utiliza para los diagramas de clases del SDK, los diagramas de componentes de la arquitectura hexagonal y los diagramas de secuencia de las interacciones entre el framework y las pasarelas.

BPMN 2.0	Se usa para los procesos de gestión del proyecto: administración de requerimientos, control de cambios, gestión de riesgos y control de calidad, todos referenciados en este documento.

Tabla 2. Lenguajes de Modelado.

3.2.2. Lenguajes de programación

Herramienta	**Justificación y aplicación en el proyecto**
TypeScript	TypeScript es el lenguaje principal del proyecto, tanto para el SDK como para la API de simulación. La elección responde a que TypeScript es hoy el estándar de facto en el ecosistema JavaScript para proyectos de biblioteca: el tipado estático mejora la experiencia del desarrollador que consume el SDK (autocompletado, verificación en tiempo de compilación) y facilita la definición explícita de los contratos de los puertos de la arquitectura hexagonal. Adicionalmente, el ecosistema npm es el más amplio para distribución de librerías de software, y TypeScript compila a JavaScript, lo que garantiza compatibilidad con entornos Node.js y plataformas frontend modernas.

Tabla 3. Lenguajes de Programación.

3.2.3. Herramientas de desarrollo y arquitectura

Herramienta	**Justificación y aplicación en el proyecto**
Node.js	A través de Fastify, node.js actúa como servidor de la API de simulación. Fastify fue preferido sobre Express por su mejor desempeño y su soporte nativo de TypeScript con tipado explícito de rutas y esquemas. Jest es el framework de pruebas para el SDK y la API de simulación; se utiliza para pruebas unitarias sobre el núcleo y pruebas de integración sobre los adaptadores. Supertest complementa Jest para las pruebas de los endpoints de la API de simulación.
Github	Mediante GitHub Actions constituye el sistema de control de versiones y la plataforma de integración continua. Cada pull request hacia la rama principal

	activa automáticamente la ejecución de las pruebas y la verificación de cobertura. npm es el gestor de paquetes y el canal de distribución del SDK como librería open source.

Tabla 4. Herramientas de desarrollo y arquitectura

3.2.4. Herramientas de análisis, modelado y documentación

Herramienta	**Justificación y aplicación en el proyecto**
Draw.io	Draw.io es la herramienta principal para la creación de diagramas UML y BPMN. Todos los diagramas del proyecto se elaboran aquí y se exportan como imagen para su inclusión en los documentos.
TypeDoc	Esta herramienta automáticamente la documentación de la API pública del SDK a partir de los comentarios JSDoc en el código fuente; esta documentación se publica en el repositorio como referencia para desarrolladores externos.
Notion	En esta plataforma se almacena la documentación viva del proyecto: actas de reunión, decisiones técnicas, ADRs y bitácoras de avance.

Tabla 5. Herramientas de análisis, modelado y documentación

3.2.5. Herramientas de gestión del proyecto y comunicación

Herramienta	**Justificación y aplicación en el proyecto**
Github Projects	Se utiliza para la gestión del backlog y el seguimiento de tareas por fase del proyecto. La integración nativa con el repositorio facilita la trazabilidad entre tareas e issues de código.
Microsoft Teams	Medio de contacto para la comunicación formal (reuniones con el director, revisiones de entregables).
Whatsapp	Coordinación diaria entre los integrantes.
Postman	Se utiliza para documentar y ejecutar colecciones de pruebas sobre los endpoints de la API de

	simulación; las colecciones se versionan en el repositorio.

Tabla 6. Herramientas de gestión del proyecto y comunicación

3.2.6. Herramientas de ofimática y presentación

Herramienta	**Justificación y aplicación en el proyecto**
Google Docs	Se usa para los documentos entregables del trabajo de grado.
Google Spreadsheets	Se usa para la carta Gantt y la matriz de métricas CK.
Draw.io	Se usa para los diagramas mostrados en los documentos.

Tabla 7. Herramientas de análisis, modelado y documentación

3.2.7. Análisis de Alternativas y Justificación

Lenguaje del SDK:

La alternativa más seria a TypeScript es Java. Las métricas CK de Chidamber y Kemerer fueron originalmente diseñadas para sistemas orientados a objetos Java, y existen herramientas consolidadas como CKJMExtended que calculan WMC, CBO y RFC directamente sobre bytecode. Una librería Java tendría menor adopción potencial y quedaría fuera del alcance del ecosistema npm, que es el canal de distribución más natural para un SDK de este tipo.

Python fue considerado brevemente por su popularidad en scripting y análisis de datos, pero no es un lenguaje típico para SDKs de integración de pagos en entornos web colombianos y el ecosistema PyPI tiene menor penetración en ese contexto.

La decisión de TypeScript implica que el cálculo de métricas CK se hace con ts-morph en lugar de herramientas Java consolidadas, lo que requiere implementar extractores de métricas personalizados.

API de simulación:

Express es la alternativa más conocida en el ecosistema Node.js, con amplia documentación y comunidad. Se descartó en favor de Fastify por dos razones: Fastify tiene soporte nativo para TypeScript con validación de esquemas mediante JSON Schema, lo que permite definir los contratos de los endpoints de la API de simulación de forma explícita y verificable; y su rendimiento es significativamente mejor que Express para el mismo código, lo que es relevante si la API de simulación necesita manejar múltiples requests concurrentes durante las pruebas. Spring Boot fue descartado por introducir una dependencia de Java en un proyecto cuyo SDK es TypeScript, generando un stack tecnológico heterogéneo innecesario.

Framework de pruebas:

Jest es el framework de pruebas más extendido en el ecosistema TypeScript/Node.js, con soporte nativo de cobertura de código, mocks integrados y una API coherente para pruebas unitarias e integración. Vitest, una alternativa más reciente compatible con Vite, ofrece mejor velocidad de ejecución pero tiene un ecosistema todavía en maduración. Mocha con Chai requiere configurar por separado el framework de aserciones, los simulacros y la cobertura, lo que añade complejidad de configuración sin beneficios sustanciales. Jest es la elección más directa y con menos fricción en la configuración.

CI/CD

Jenkins requiere servidor propio, configuración manual y mantenimiento continuo, lo que representa una carga operativa representativa. GitHub Actions se integra nativamente con el repositorio sin infraestructura adicional; los workflows se definen como archivos YAML versionados y su configuración para proyectos Node.js es directa. La elección de GitHub Actions también elimina la necesidad de una cuenta o servidor adicional.

Gestión de backlog

Jira es la herramienta más completa, pero tiene una curva de configuración alta y sus planes gratuitos tienen limitaciones para equipos que necesitan integración con GitHub. Linear es una alternativa moderna con buena integración con GitHub, pero introduce una herramienta externa adicional. GitHub Projects se integra directamente con los issues y pull requests del repositorio, lo que facilita la trazabilidad entre tareas y código sin salir del entorno de trabajo. Para un proyecto de esta escala, la integración nativa supera la funcionalidad adicional que ofrecen las alternativas.

3.3. Plan de Aceptación del Producto

Los criterios de aceptación de cada entregable son acordados entre el equipo y el director desde el inicio del proyecto. Todos los criterios deben ser verificables objetivamente; ninguno depende de apreciaciones subjetivas del evaluador.

3.3.1. Prototipo funcional del SDK

El SDK se considera aceptado cuando compila sin errores en TypeScript estricto, cuando los cuatro adaptadores (Wompi, PayU, Mercado Pago, Kushki) producen respuestas normalizadas correctas ante los cuatro escenarios de prueba definidos en la API de simulación, cuando la cobertura de pruebas unitarias sobre el núcleo es igual o superior al 80%, y cuando el paquete puede instalarse desde npm y usarse en un proyecto externo nuevo sin modificar su código fuente. La herramienta de validación es Jest para las pruebas automatizadas y npm pack para la verificación de instalación.

3.3.2. API de simulación desplegada

La API se considera aceptada cuando todos los endpoints documentados responden en menos de 500ms bajo condiciones normales, cuando cubre los cuatro escenarios de prueba para cada pasarela (pago exitoso, rechazo, error de red y notificación webhook), cuando está desplegada y accesible desde internet sin necesidad de credenciales reales, y cuando la colección de Postman que documenta sus endpoints produce resultados exitosos al ejecutarse desde un entorno limpio. La herramienta de validación es Postman y el registro de logs del servidor de despliegue.

3.3.3. Documentación centralizada de datos de prueba

La documentación se considera aceptada cuando cubre las cuatro pasarelas con al menos los siguientes datos por proveedor: tarjetas de prueba con sus CVV y fechas de vencimiento, documentos de identidad válidos para sandbox y referencias de pago de ejemplo. Debe estar publicada en el repositorio público y validada contra el comportamiento real del sandbox de cada pasarela mediante al menos una prueba manual documentada por pasarela.

3.3.4. Informe comparativo de métricas CK

El informe se considera aceptado cuando contiene los valores de WMC, CBO y RFC calculados con ts-morph sobre las mismas clases en ambos escenarios (integración directa e integración con el framework), cuando los proyectos prototípicos sobre los que se calculan están documentados con suficiente detalle para que el experimento sea reproducible, y cuando el análisis comparativo incluye una interpretación de las diferencias observadas en términos de complejidad y acoplamiento. El director valida la pertinencia del análisis.

3.3.5. Documentación final del trabajo de grado y repositorio público

La documentación final se considera aceptado cuando cubre todas las fases del ciclo DSR con sus resultados, cuando están todos los documentos requeridos (Memoria, SAD, SDD, SPMP, SRS, VFP) cuando el repositorio público incluye instrucciones de instalación, ejemplos de integración para cada pasarela y la documentación generada por TypeDoc, y cuando un integrante no involucrado en la escritura de una sección puede seguir las instrucciones delrepositorio e integrar el SDK en un proyecto nuevo sin contactar a los autores. El director aprueba formalmente el documento.

3.4. Organización del Proyecto y Comunicación

La estructura de participantes del Kit Pagos Colombia es más acotada que la de proyectos con clientes externos o múltiples grupos de interés. El equipo de desarrollo no negocia entregas con un stakeholder ni valida funcionalidades con usuarios en campo.

El proyecto tiene adicionalmente dependencias técnicas con las cuatro pasarelas de pago analizadas en la fase de identificación. Estas entidades no intervienen activamente en el proceso; la relación con ellas es unidireccional: el equipo consume sus portales de documentación y entornos sandbox, sin coordinación humana directa bajo condiciones normales.

3.4.1. Interfaces Externas

Nombre de la entidad	Descripción	Responsabilidades en el proyecto	Datos de contacto / medio de comunicación.
Ph.D. Jaime Andrés Pavlich Mariscal	Director del trabajo de grado. Profesor titular del Departamento de Ingeniería de Sistemas de la Pontificia Universidad Javeriana	Supervisión de la ejecución teniendo en cuenta lo establecido en la metodología. Aprobación de entregables clave; retroalimentación sobre decisiones	jpavlich@javeriana.edu.co / Ext. 4731 / Reuniones presenciales o virtuales acordadas durante el semestre.

		arquitectónicas y criterios de evaluación; lineamientos académicos generales.	
Pontificia Universidad Javeriana	Institución académica en la que se desarrolla el proyecto	Marco institucional del trabajo de grado; derechos de uso académico sobre el documento final; acceso a recursos y plataformas institucionales	Canales institucionales de la universidad
Wompi (Bancolombia)	Pasarela de pago colombiana operada por Bancolombia, uno de los cuatro proveedores integrados en el framework	Provee documentación técnica de su API REST, entorno sandbox y datos de prueba utilizados en la fase de análisis, desarrollo y evaluación	docs.wompi.co
PayU Latam	Pasarela de pago con operación activa en Colombia, uno de los cuatro proveedores integrados en el framework	Provee documentación técnica de su API, entorno sandbox y datos de prueba.	developers.payulatam.com
Mercado Pago	Pasarela de origen argentino con operación en Colombia y varios países de latinoamérica, uno de los cuatro proveedores integrados en el framework	Provee documentación técnica de su API, entorno sandbox y datos de prueba	developers.mercadopago.com
Kushki	Plataforma latinoamericana con operación activa en Colombia, uno de los cuatro proveedores integrados en el framework	Provee documentación técnica de su API, entorno sandbox y datos de prueba	docs.kushkipagos.com

Tabla 8. Interfaces Externas

Toda interacción relevante con estas entidades quedará registrada en actas o bitácoras. La comunicación con las pasarelas de pago ocurre exclusivamente a través de sus portales públicos; el contacto directo con equipos de soporte se reserva para situaciones que lo justifiquen y que no puedan resolverse por documentación.

3.4.2. Organigrama y Descripción de Roles

A diferencia de esquemas con rotación de roles por sprint, se adoptaron asignaciones estables que reflejan las especializaciones técnicas del framework: un componente de arquitectura y núcleo, cuatro adaptadores de pasarela con una API de simulación, y un proceso de evaluación con métricas de código.

Cada integrante lidera uno de esos frentes sin dejar de contribuir a los demás, especialmente durante la fase de desarrollo iterativo, es decir que todos deben aportar a cada entregable de manera equitativa, independientemente su responsabilidad principal dentro de lo establecido a continuación.

La jerarquía en el nivel operativo es plana. El coordinador del equipo no tiene autoridad técnica sobre los demás sino responsabilidad sobre la gestión del backlog, el calendario y la comunicación con el director. Las decisiones técnicas se toman por consenso entre los integrantes involucrados en el frente correspondiente.

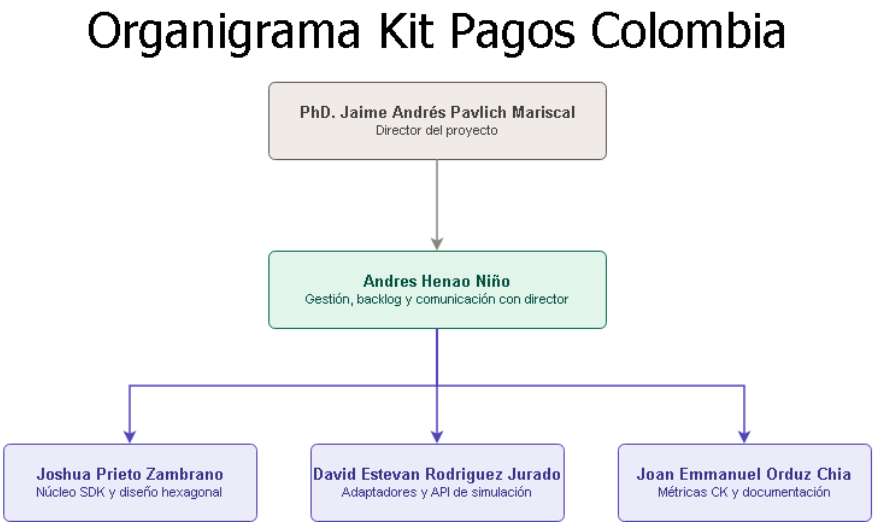


Figura 2. Organigrama del equipo de trabajo.

Integrante	**Rol**	**Descripción**	**Responsabilidades**
Ph.D. Jaime Andrés Pavlich Mariscal	Director del Proyecto	Supervisor académico externo al equipo. No participa en la implementación técnica	Aprobación de entregables clave, retroalimentación sobre arquitectura, métricas de evaluación y alcance del framework
Andres Henao Niño	Coordinador del equipo	Integrante responsable de la gestión operativa del proyecto. Participa activamente en el desarrollo	Mantenimiento del backlog y seguimiento del calendario; coordinación de reuniones internas; comunicación con el director; registro de actas y decisiones clave

Joshua Prieto Zambrano	Líder de arquitectura y núcleo SDK	Responsable del diseño estructural del framework y la implementación del núcleo	Diseño del modelo de dominio unificado; especificación de los puertos de la arquitectura hexagonal; implementación del núcleo del SDK; revisión de que los adaptadores cumplan los contratos de puerto
David Estevan Rodriguez Jurado	Líder de integración	Responsable de los cuatro adaptadores por pasarela y de la API de simulación	Implementación de los adaptadores de Wompi, PayU, Mercado Pago y Kushki; desarrollo y despliegue de la API de simulación; consolidación de la documentación de datos de prueba
Joan Emmanuel Orduz Chia	Líder de evaluación y documentación	Integrante responsable del proceso de medición con métricas CK, los proyectos prototípicos y el documento final	Diseño y ejecución de la evaluación comparativa con métricas WMC, CBO y RFC; desarrollo de los prototipos de medición; redacción del documento de grado y publicación del repositorio público

Tabla 9. Descripción de los roles.

Los canales de comunicación internos del equipo son los siguientes: para coordinación diaria se utiliza WhatsApp; para reuniones formales de planificación y revisión se usa la herramienta de videoconferencia Microsoft Teams; para la gestión del backlog y seguimiento de tareas se usa Google Docs; y para el control de versiones y revisión de código se usa GitHub con política de pull requests para cambios sobre la rama principal.

4. Administración del Proyecto

La administración del proyecto define los mecanismos utilizados para planificar, coordinar y controlar la ejecución del Kit Pagos Colombia a lo largo de todas sus fases. Esta sección describe cómo se estimará el esfuerzo, cómo se pondrá en marcha el entorno de trabajo y cómo se organizarán las actividades, recursos y responsabilidades necesarias para garantizar el cumplimiento de los objetivos del proyecto.

4.1. Métodos y Herramientas de Estimación

La estimación del proyecto se realizó sobre dos dimensiones: el tamaño del software y el esfuerzo requerido para producirlo. Las estimaciones de tamaño y esfuerzo se consolidaron inicialmente en Google Spreadsheets, utilizando tablas de descomposición de componentes y distribución de carga por fase.

GitHub Projects se utilizó complementariamente para hacer seguimiento al avance real frente a las estimaciones iniciales durante la ejecución del proyecto.

4.1.1. Estimación del tamaño del software

Se empleó estimación por conteo de componentes, partiendo de la arquitectura ya definida en la Fase 3. Este método consiste en identificar los módulos principales del sistema, asignarles una complejidad relativa y derivar de ahí un volumen de trabajo estimado en clases y módulos TypeScript.

Componente	Descripción	Clases/módulos estimados
Núcleo del SDK	Modelo de dominio, interfaces de puerto, lógica de normalización	8–12
Adaptador Wompi	Implementación de puertos para Wompi	4–6
Adaptador PayU	Implementación de puertos para PayU	4–6
Adaptador Mercado Pago	Implementación de puertos para Mercado Pago	4–6
Adaptador Kushki	Implementación de puertos para Kushki	4–6
API de simulación	Controladores Fastify, manejadores de escenarios	8–12
Infraestructura de pruebas	Suites Jest para núcleo, adaptadores y API	10–15
Total		42–63

Estimación del esfuerzo del proyecto

Se combinaron juicio de expertos y estimación por analogía. El referente es el trabajo de Manuaba e Hisyam (2022), que integró dos pasarelas de pago en condiciones similares. El Kit Pagos Colombia integra cuatro pasarelas, añade la API de simulación y el componente de evaluación con métricas CK, lo que representa aproximadamente el doble de la complejidad del referente.

El aumento en la complejidad frente al referente no se debe únicamente a que el proyecto integra cuatro pasarelas en lugar de dos. El framework también incluye componentes adicionales que incrementan el esfuerzo de desarrollo, como la API de simulación, las pruebas automatizadas, la documentación técnica generada para el SDK y el proceso de evaluación mediante métricas CK. Además, la arquitectura hexagonal adoptada requiere definir previamente los contratos y la estructura del núcleo antes de implementar los adaptadores, lo que añade trabajo de diseño y validación técnica durante las primeras fases del proyecto.

1. Cada integrante destina en promedio 12 horas semanales durante las 18 semanas del

semestre, resultando en 864 horas-persona de esfuerzo total. La distribución por fase es la siguiente:

Fase DSR	Porcentaje	Horas estimadas
Identificación del problema	10 %	87
Definición de objetivos	8 %	69
Diseño	12 %	104
Desarrollo iterativo	45 %	389
Demostración y evaluación	15 %	130
Comunicación	10 %	85
Total	100 %	864

El seguimiento del esfuerzo real frente al estimado se realiza en Google Spreadsheets, donde cada integrante registra sus horas al cierre de cada semana y el coordinador las compara con las horas proyectadas por fase. La tasa de completitud del backlog en GitHub Projects actúa como indicador proxy del avance: si el porcentaje de ítems completados es coherente con las horas consumidas, la estimación se considera válida. Una desviación superior al 20% entre esfuerzo real y proyectado activa una revisión en la reunión semanal.

4.2. Inicio del proyecto

El inicio del proyecto contempla las actividades necesarias para preparar al equipo y al entorno de trabajo antes del desarrollo del framework. Esto incluye la transferencia de conocimientos técnicos, la organización de herramientas y la configuración de la infraestructura requerida para asegurar una ejecución consistente durante las diferentes fases del proyecto.

4.2.1. Entrenamiento del Personal

El equipo cuenta con base sólida en programación orientada a objetos, TypeScript y consumo de APIs REST. No obstante, la ejecución del proyecto requiere competencias específicas que no todos los integrantes dominan con la misma profundidad al inicio. El plan a continuación cierra esas brechas.

Conocimiento / Habilidad	Brecha identificada	Integrantes	Responsable de la transferencia	Mecanismo	Momento

Arquitectura hexagonal aplicada en TypeScript	Conocimiento conceptual presente; aplicación práctica insuficiente	Todos	Líder de arquitectura y núcleo	Sesión grupal de 2 horas con código de ejemplo; revisión del capítulo aplicado de Cockburn (2024)	Inicio de la Fase 3
Fastify con TypeScript y validación de esquemas JSON	Conocimiento de Express presente; Fastify desconocido	Líder de integración	Autoaprendizaje guiado con documentación oficial de Fastify; endpoint de ejemplo documentado en Notion	Fase 3	
ts-morph para extracción de métricas CK	Herramienta no conocida por ningún integrante	Líder de evaluación (autoaprendizaje previo) → sesión de transferencia	Autoaprendizaje individual; sesión práctica grupal de 1.5 horas con casos reales del proyecto	Autoaprendizaje en Fase 3; sesión grupal antes de Fase 5	
APIs de las cuatro pasarelas	Conocimiento heterogéneo; ningún integrante conoce las cuatro	Un integrante por pasarela según asignación de la Fase 1	Revisión documental individual de documentación oficial; transferencia al equipo mediante programación en pares en Fase 4	Fase 1; transferencia en Fase 4	
Publicación de paquete en npm	No realizado previamente por el equipo	Líder de arquitectura	Autoaprendizaje con documentación oficial de npm; guía de publicación redactada en Notion	Antes del hito H3	

El mecanismo transversal de transferencia es una sesión técnica de 30 minutos al inicio de cada semana de trabajo, integrada a la revisión del backlog, en la que cada integrante comparte lo aprendido durante la semana anterior. Los artefactos resultantes (guías, fragmentos de código, decisiones técnicas) se documentan en Notion para que estén accesibles durante todo el proyecto. La transferencia de conocimiento se considera completada cuando los integrantes pueden aplicar de manera autónoma lo aprendido en actividades reales del proyecto, como la implementación de componentes, configuración de herramientas o ejecución de pruebas relacionadas con su frente de trabajo.

4.2.2. Infraestructura

La siguiente tabla describe todas las tareas de configuración y mantenimiento de la infraestructura del proyecto, con su responsable, momento de ejecución y frecuencia.

Tarea	Rol responsable	Momento	Frecuencia
Crear repositorio GitHub con estructura de ramas (main, develop, feature/*) y política de protección de rama principal	Líder de arquitectura	Semana 1	Una vez; política se revisa si surgen problemas de flujo
Inicializar proyecto TypeScript con tsconfig estricto, estructura de paquete npm y licencia Apache 2.0	Líder de arquitectura	Semana 1	Una vez; tsconfig se ajusta al agregar dependencias
Configurar pipeline de GitHub Actions (lint con ESLint, pruebas con Jest, reporte de cobertura)	Coordinador	Semana 2	Una vez; actualizar al agregar nuevas fases al pipeline
Crear tablero en GitHub Projects con columnas Backlog / En progreso / Completado; cargar backlog inicial	Coordinador	Semana 1	Backlog se actualiza al inicio de cada fase
Crear cuenta de despliegue en Railway o Render; configurar despliegue continuo desde la rama main	Líder de integración	Inicio Fase 4, Iteración 3	Una vez; verificar disponibilidad mensualmente
Configurar espacio de trabajo en Notion (actas, ADRs, bitácoras, registro de cambios)	Coordinador	Semana 1	Actualización continua por todos los integrantes
Verificar acceso a los entornos sandbox de las cuatro pasarelas (credenciales, disponibilidad de endpoints)	Líder de integración	Inicio Fase 1 e inicio Fase 4	Al inicio de cada fase de desarrollo
Configurar colección inicial en Postman con variables de entorno para sandbox y para API de simulación	Líder de integración	Inicio Fase 4, Iteración 3	Actualizar por cada endpoint nuevo implementado

Configurar TypeDoc para generación automática de documentación a partir de comentarios JSDoc	Líder de arquitectura	Inicio Fase 4, Iteración 1	Verificar en cada release
Publicar versión inicial del paquete en npm (0.1.0, marcada como pre-release)	Líder de arquitectura	H3 completado	Por cada release

Todas las credenciales (tokens de sandbox, claves de despliegue) se almacenan como secretos en GitHub Actions y nunca se versionan en el repositorio. El integrante que crea una credencial es responsable de documentar su propósito en Notion sin exponer el valor.

4.3. Planes de Trabajo del Proyecto

Los planes de trabajo describen cómo se ejecutarán las principales actividades del proyecto durante su desarrollo. Esta sección define la descomposición de tareas, la calendarización, la asignación de recursos y la distribución del esfuerzo, estableciendo responsables, herramientas y momentos de ejecución para cada actividad relevante del proyecto.

4.3.1. Descomposición de Actividades

La estructura de descomposición de trabajo (WBS) del proyecto se construyó tomando como referencia las seis fases del enfoque Design Science Research (DSR) definido para el desarrollo del framework. Cada fase agrupa actividades técnicas, organizacionales y de validación asociadas a los principales entregables del proyecto.

Dado el carácter iterativo de la Fase 4 —correspondiente al desarrollo del artefacto— se presenta el detalle completo de una iteración representativa, incluyendo actividades de implementación, pruebas y validación incremental de los componentes del SDK. La gestión del proyecto se considera transversal a todas las fases y contempla actividades permanentes de seguimiento, documentación y control del avance.

1. Kit Pagos Colombia

1.1 Gestión del proyecto (transversal)

- 1.1.1 Elaboración y mantenimiento del SPMP
- 1.1.2 Seguimiento semanal del backlog
- 1.1.3 Reportes quincenales de progreso al director
- 1.1.4 Registro de actas de reunión y decisiones clave

1.2 Fase 1 – Identificación del problema

- 1.2.1 Análisis de autenticación, respuestas y webhooks de Wompi
- 1.2.2 Análisis de autenticación, respuestas y webhooks de PayU
- 1.2.3 Análisis de autenticación, respuestas y webhooks de Mercado Pago
- 1.2.4 Análisis de autenticación, respuestas y webhooks de Kushki
- 1.2.5 Consolidación del mapa comparativo del ecosistema (Hito H1)

- 1.2.6 Repositorio de parámetros de prueba por pasarela (Hito H1)

1.3 Fase 2 – Definición de objetivos

- 1.3.1 Documento de alcance formal (SRS inicial)
- 1.3.2 Criterios de evaluación formalizados
- 1.3.3 Definición y documentación de los proyectos prototípicos

1.4 Fase 3 – Diseño

- 1.4.1 Modelo de dominio unificado (clases y relaciones)
- 1.4.2 Especificación de puertos de la arquitectura hexagonal
- 1.4.3 Especificación de adaptadores por pasarela
- 1.4.4 Diseño de la API de simulación (endpoints y escenarios)
- 1.4.5 Registro de ADRs para decisiones arquitectónicas clave (Hito H2)

1.5 Fase 4 – Desarrollo iterativo

- 1.5.1 Iteración 1 – Núcleo del SDK
- 1.5.1.1 Implementación del modelo de dominio y contratos de puerto
- 1.5.1.2 Pruebas unitarias del núcleo (cobertura ≥ 80 %)
- 1.5.1.3 Revisión técnica grupal del núcleo (Hito H3 parcial)

1.5.2 Iteración 2 – Adaptadores de pasarela

- 1.5.2.1 Desarrollo del adaptador Wompi con pruebas de integración
- 1.5.2.2 Desarrollo del adaptador PayU con pruebas de integración
- 1.5.2.3 Desarrollo del adaptador Mercado Pago con pruebas de integración
- 1.5.2.4 Desarrollo del adaptador Kushki con pruebas de integración
- 1.5.2.5 Validación cruzada de los cuatro adaptadores (Hito H3 completo)

1.5.3 Iteración 3 – API de simulación y soporte de validación

- 1.5.3.1 Implementación de escenarios de prueba para las pasarelas soportadas
- 1.5.3.2 Despliegue de la API en Railway o Render
- 1.5.3.3 Documentación centralizada de datos de prueba
- 1.5.3.4 Colección Postman versionada en el repositorio (Hito H4)

1.6 Fase 5 – Demostración y evaluación

- 1.6.1 Construcción de prototipos con integración directa a APIs nativas
- 1.6.2 Construcción de prototipos utilizando el framework desarrollado
- 1.6.3 Cálculo de métricas WMC, CBO y RFC mediante ts-morph
- 1.6.4 Elaboración del informe comparativo (Hito H5)

1.7 Fase 6 – Comunicación

- 1.7.1 Redacción y revisión del documento final de trabajo de grado
- 1.7.2 Publicación del repositorio público con README, ejemplos y TypeDoc
- 1.7.3 Entrega formal y presentación ante director y evaluadores (Hito H6)

4.3.2. Calendarización

CARTA GANTT — Kit Pagos Colombia | SPMP

Pontificia Universidad Javeriana · Facultad de Ingeniería · Trabajo de Grado · Mayo 2026

WBS	Actividad	Responsable	Hito	S01	S02	S03	S04	S05	S06	S07	S08	S09	S10	S11	S12	S13	S14	S15	S16	S17	S18
1	**Kit Pagos Colombia**	**Equipo**																			
1.1	Gestión del Proyecto (Transversal)	Coordinador																			
1.1.1	Elaboración y mantenimiento del SPMP	Coordinador																			
1.1.2	Seguimiento semanal del backlog	Coordinador																			
1.1.3	Reportes quincenales al director	Coordinador																			
1.1.4	Registro de actas y decisiones	Coordinador																			
1.2	**Fase 1 – Identificación del Problema**	**Equipo**																			
1.2.1	Análisis técnico Wompi	Líder Integración Wompi																			
1.2.2	Análisis técnico PayU	Líder Integración PayU																			
1.2.3	Análisis técnico Mercado Pago	Líder Integración Mercado P																			
1.2.4	Análisis técnico Kushki	Líder Integración Kushki																			
1.2.5	Consolidacion mapa comparativo ecosistema	Coordinador	H1																		
1.2.6	Repositorio de parámetros de prueba	Coordinador	H1			▲															
1.3	**Fase 2 – Definición de Objetivos**	**Equipo**																			
1.3.1	Documento de alcance formal (SRS inicial)	Líder Evaluación																			
1.3.2	Criterios de evaluación formalizados	Líder Evaluación																			
1.3.3	Definición de proyectos prototípicos	Equipo																			
1.4	**Fase 3 – Diseño**	**Líder Arquitectura**																			
1.4.1	Modelo de dominio unificado	Líder Arquitectura																			
1.4.2	Especificación de puertos hex. architecture	Líder Arquitectura																			
1.4.3	Especificación de adaptadores por pasarela	Líder Integración																			
1.4.4	Diseño API de simulación y escenarios	Líder Integración																			
1.4.5	Registro de ADRs decisiones arquitectónicas	Líder Arquitectura	H2																		
1.5	**Fase 4 – Desarrollo Iterativo**	**Equipo**																			
1.5.1	Iteración 1 – Núcleo del SDK	Líder Arquitectura								▲											
1.5.1.1	Implementación modelo dominio y puertos	Líder Arquitectura																			
1.5.1.2	Pruebas unitarias del núcleo (≥80%)	Líder Arquitectura																			
1.5.1.3	Revisión técnica grupal del núcleo	Equipo	H3*																		
1.5.2	Iteración 2 – Adaptadores de Pasarela	Líder Integración										▲									
1.5.2.1	Adaptador Wompi + pruebas integración	Líder Integración																			
1.5.2.2	Adaptador PayU + pruebas integración	Líder Integración																			
1.5.2.3	Adaptador Mercado Pago + pruebas	Líder Integración																			
1.5.2.4	Adaptador Kushki + pruebas integración	Líder Integración																			
1.5.2.5	Validación cruzada cuatro adaptadores	Equipo	H3																		
1.5.3	Iteración 3 – API Simulación y Validación	Líder Integración																			
1.5.3.1	Impl. escenarios prueba por pasarela	Líder Integración													▲						
1.5.3.2	Despliegue API en Railway/Render	Líder Integración																			
1.5.3.3	Documentacion centralizada datos de prueba	Líder Integración																			
1.5.3.4	Coleccion Postman versionada en repositorio	Líder Integración	H4																		
1.6	**Fase 5 – Demostración y Evaluación**	**Líder Evaluación**																			
1.6.1	Prototipos integración directa APIs nativas	Equipo														▲					
1.6.2	Prototipos utilizando el framework	Equipo																			
1.6.3	Cálculo métricas WMC, CBO, RFC (ts-morph)	Líder Evaluación																			
1.6.4	Elaboración informe comparativo métricas CK	Líder Evaluación	H5																		
1.7	**Fase 6 – Comunicación**	**Líder Evaluación**																	▲		
1.7.1	Redacción y revisión documento final grado	Equipo																			
1.7.2	Publicación repositorio público + TypeDoc	Líder Arquitectura																			
1.7.3	Entrega formal y presentación director	Coordinador	H6																		
																					▲

LEYENDA

Gestión (transversal) · Fase 1 – Identif · Fase 2 – D · Fase 3 – D · Fase 4 – Ite · Fase 4 – Ite · Fase 4 – Ite · Fase 5 – E · Fase 6 – C · ▲ Hito

Figura 3. Carta Gantt

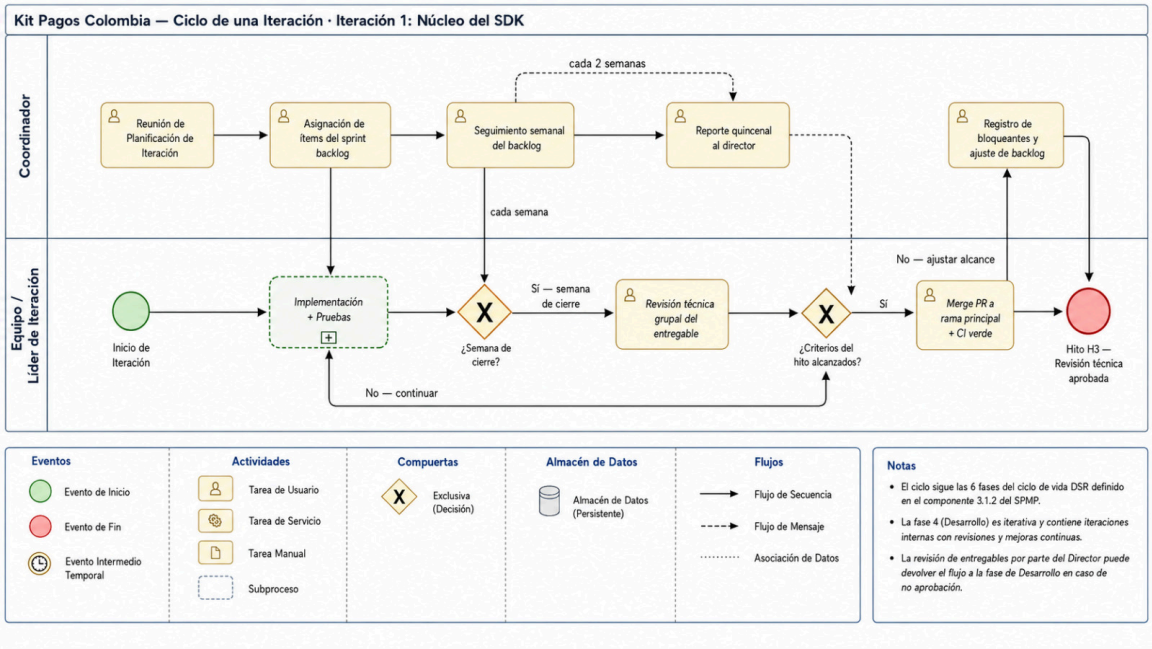


Figura 4. Diagrama BPMN iterativo

4.3.3. Asignación de Recursos

Para cada actividad principal del WBS se indica el conjunto de roles involucrados y las herramientas necesarias para ejecutarla.

Actividad (WBS)	Roles involucrados	Herramientas
1.1 Gestión del proyecto	Coordinador (primario); todos los integrantes como reportantes	GitHub Projects, Notion, Microsoft Teams, Google Docs
1.2 Fase 1 – Identificación	Todos (un integrante por pasarela); Coordinador consolida el mapa del ecosistema	Documentaciones oficiales de las pasarelas, Notion, Draw.io
1.3 Fase 2 – Definición	Líder de evaluación redacta el SRS; Coordinador gestiona el backlog; todos revisan y aprueban	Google Docs, GitHub Projects, Notion
1.4 Fase 3 – Diseño	Líder de arquitectura lidera; todos revisan los contratos de puerto y los ADRs	Draw.io (UML y BPMN), Notion (ADRs), Google Docs
1.5.1 Iter 1 – Núcleo del SDK	Líder de arquitectura (primario); revisión obligatoria de todos antes de iniciar Iter 2	TypeScript, Jest, GitHub (PR + CI), VS Code
1.5.2 Iter 2 – Adaptadores	Líder de integración coordina; cada adaptador se desarrolla en par con otro integrante	TypeScript, Jest, sandboxes de las cuatro pasarelas, GitHub
1.5.3 Iter 3 – API de simulación	Líder de integración (primario); Líder de arquitectura apoya en diseño de contratos	Fastify, TypeScript, Railway o Render, Postman, GitHub
1.6 Fase 5 – Evaluación	Líder de evaluación lidera; todos construyen los prototipos asignados	ts-morph, TypeScript, Jest, Google Spreadsheets

1.7 Fase 6 – Comunicación	Líder de evaluación coordina la redacción del documento; todos contribuyen por secciones; Coordinador gestiona la publicación del repositorio	Google Docs, GitHub, TypeDoc, npm

Los recursos del proyecto son exclusivamente humanos y de herramientas software. No existe hardware dedicado: cada integrante trabaja desde su equipo personal. La disponibilidad de los sandboxes de las pasarelas es un recurso externo no controlado por el equipo; su indisponibilidad prolongada activa el protocolo de riesgo descrito en la sección 7.2.

4.3.4. Asignación de Presupuesto y Justificación

Costos reales del proyecto

El proyecto opera íntegramente con herramientas de acceso gratuito. Los únicos costos potencialmente reales son los asociados al despliegue de la API de simulación.

Concepto	Período	Costo estimado (COP)	Justificación
Despliegue en Railway o Render	Semanas 10–18	$0	Ambas plataformas ofrecen tier gratuito con recursos suficientes para el volumen de pruebas del proyecto
Herramientas de desarrollo (GitHub, npm, TypeScript, Jest, Postman, Notion)	Semanas 1–18	$0	Gratuitas para proyectos open source o uso educativo
Dominio y certificado SSL	Semanas 10–18	$0	Las plataformas de despliegue proveen subdominio y HTTPS sin costo adicional
Total de costos reales	Semanas 1–18	$0	

Estimación del costo del software

Aunque no se movilizan recursos financieros reales, el esfuerzo invertido tiene un valor económico equivalente. Para su cálculo se utiliza una tarifa de referencia de COP 25.000 por hora, correspondiente al rango de mercado para un desarrollador junior con competencias en TypeScript y arquitecturas de software en Colombia en 2026.

Fase DSR	Período	Horas estimadas	Costo estimado (COP)
Identificación del problema	Semanas 1–3	87	2.175.000
Definición de objetivos	Semanas 3–5	69	1.725.000
Diseño	Semanas 5–7	104	2.600.000
Desarrollo iterativo	Semanas 7–13	389	9.725.000
Demostración y evaluación	Semanas 13–16	130	3.250.000
Comunicación	Semanas 16–18	87	2.175.000
Total	Semanas 1–18	866	21.650.000

Este cálculo tiene propósito referencial y no implica flujo real de dinero. Su utilidad es dimensionar el valor del trabajo producido y servir como punto de comparación en caso de que proyectos similares requieran contratación externa.

5. Monitoreo y Control del Proyecto

El presente numeral describe los mecanismos que el equipo de desarrollo del Kit Pagos Colombia utilizará para conocer el estado de salud del proyecto en todo momento, detectar desviaciones respecto a lo planeado y ejecutar acciones correctivas oportunas. Los procesos aquí definidos son transversales a todas las fases del ciclo de vida DSR y operan en paralelo con la ejecución técnica del proyecto.

5.1. Administración de Requerimientos

El propósito de este proceso es garantizar que cualquier cambio en los requerimientos del producto sea detectado, evaluado, trazado y controlado de forma sistemática, de modo que el equipo y el director del proyecto conozcan en todo momento el estado de los requerimientos y las implicaciones de cualquier modificación sobre el alcance, el cronograma y los entregables.

5.1.1. Artefactos del proceso

Los artefactos que se producen y mantienen como resultado de la administración de requerimientos son los siguientes.

El primero es el Documento de Especificación de Requerimientos de Software (SRS), que constituye la línea base de requerimientos del proyecto. Este documento recoge los requerimientos funcionales organizados por componente, las historias de usuario con sus criterios de aceptación y los requerimientos no funcionales bajo la clasificación FURPS+. El SRS se considera un ítem de configuración y cualquier modificación sobre él debe seguir el proceso de control de cambios descrito en la sección 7.3.

El segundo es la Matriz de Trazabilidad de Requerimientos, que vincula cada requerimiento del SRS con el o los componentes del framework que lo implementan, con las pruebas que lo verifican y con el hito del ciclo de vida al que pertenece. Esta matriz permite detectar requerimientos huérfanos, componentes sin cobertura de prueba y el impacto de un cambio sobre el resto del sistema. La herramienta de soporte es Google Spreadsheets y su actualización es responsabilidad del integrante que asume el rol de ingeniero de requerimientos en cada fase.

El tercero es el Registro de Cambios de Requerimientos, un log mantenido en Notion que registra cada solicitud de cambio recibida, su estado (pendiente, aprobada, rechazada, implementada) y la referencia al pull request o al commit que materializó el cambio aprobado.

5.1.2. Proceso de administración de requerimientos

El proceso opera bajo las siguientes reglas. Cualquier integrante del equipo o el director puede proponer una modificación a los requerimientos. La propuesta se registra en Notion con una descripción del cambio, la justificación y el impacto estimado sobre el cronograma y los entregables. El equipo evalúa la propuesta en la reunión semanal y decide por consenso si se aprueba, se rechaza o se posterga. Si el director del proyecto ha solicitado el cambio, su aprobación es requisito necesario antes de proceder. Un cambio aprobado actualiza el SRS, la Matriz de Trazabilidad y el backlog de GitHub Projects en ese orden. Ningún cambio sobre la rama principal del repositorio se acepta sin que el SRS esté actualizado.

El diagrama BPMN del proceso de administración de requerimientos, que incluye los roles participantes, los puntos de decisión y los artefactos que se crean o refinan en cada paso, se presenta en la Figura 2.

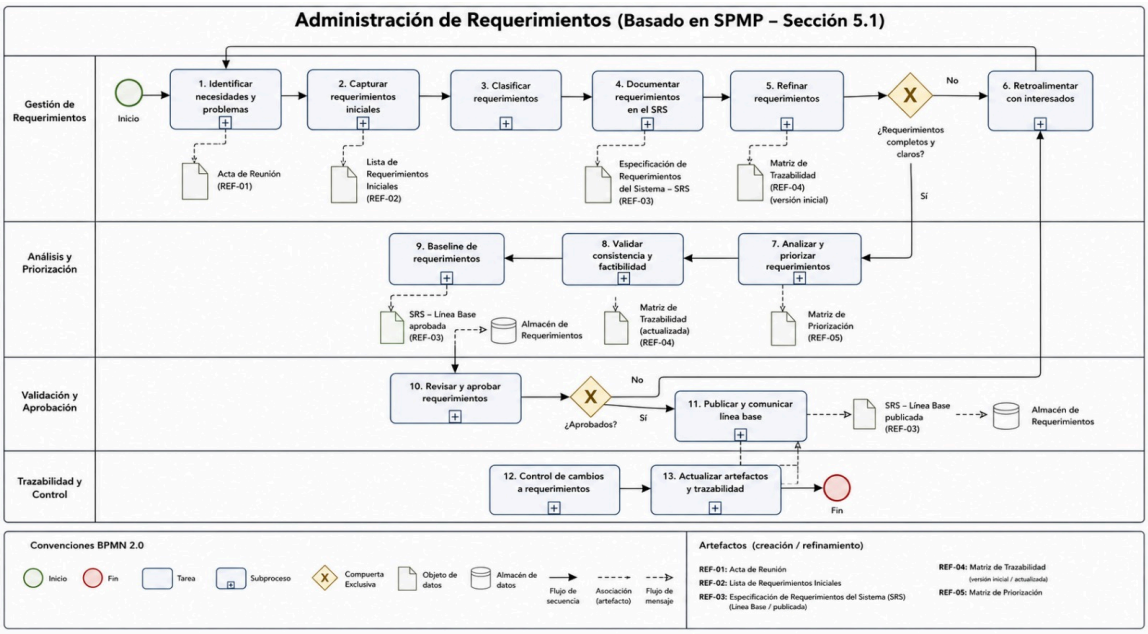


Figura 5. Diagrama BPMN adminstración de requerimientos

5.2. Monitoreo y Control de Progreso

El propósito de este proceso es proveer al equipo y al director del proyecto de información objetiva y periódica sobre el avance real del proyecto frente al plan establecido, y definir las acciones correctivas que se ejecutarán cuando el progreso no sea el esperado.

5.2.1. Unidades de medición del progreso

El progreso del proyecto se mide a través de las siguientes métricas.

La tasa de completitud del backlog se expresa como el porcentaje de ítems del backlog de GitHub Projects marcados como completados respecto al total de ítems planificados para la fase en curso. Un ítem se considera completado cuando su pull request asociado ha sido aprobado y mergeado a la rama principal. Esta métrica se revisa semanalmente.

El estado de los hitos del ciclo de vida es una métrica binaria que indica si cada uno de los seis hitos definidos en la sección 3.1.3 ha sido alcanzado o no en la fecha comprometida. Los hitos son los puntos de referencia más gruesos del proyecto y su incumplimiento activa el protocolo de acción correctiva.

La cobertura de pruebas del núcleo del SDK se expresa como el porcentaje de líneas de código del núcleo cubiertas por pruebas unitarias automatizadas, reportado por Jest al finalizar cada ejecución de la suite de pruebas en el pipeline de integración continua de GitHub Actions. El umbral mínimo aceptable es del 80%, conforme a lo establecido en el Plan de Aceptación (sección 3.3.1).

El tiempo de respuesta de la API de simulación se mide en milisegundos desde el envío de la solicitud hasta la recepción de la respuesta completa, utilizando Postman o k6 desde una red estándar. Esta métrica se verifica en cada iteración de la fase de desarrollo y debe mantenerse por

debajo de los 500ms para los escenarios sin latencia intencionada.

El número de defectos abiertos por componente se registra en GitHub Issues y se clasifica por componente (núcleo, adaptador Wompi, adaptador PayU, adaptador Mercado Pago, adaptador Kushki, API de simulación) y por severidad (bloqueante, mayor, menor). Esta métrica se revisa semanalmente.

5.2.2. Actividades de reporte de progreso

El progreso se reporta mediante las siguientes actividades periódicas.

La revisión semanal de backlog se realiza cada lunes al inicio de la semana de trabajo. Los cuatro integrantes actualizan el estado de sus ítems en GitHub Projects, registran los impedimentos encontrados y estiman el trabajo pendiente para la semana. La duración máxima de esta actividad es de 30 minutos. El responsable de convocarla es el integrante que asume el rol de coordinador en cada fase.

El reporte de progreso al director se entrega quincenalmente por correo electrónico o mediante reunión en Microsoft Teams, según la disponibilidad del director. El reporte incluye el estado de los hitos, la tasa de completitud del backlog, el estado de la cobertura de pruebas y los defectos abiertos por componente. La elaboración del reporte es responsabilidad del coordinador de la fase en curso.

La demostración al final de cada iteración de desarrollo se realiza internamente entre los cuatro integrantes al cierre de cada iteración de la fase de desarrollo (núcleo, adaptadores, API de simulación). En esta instancia se verifica que los criterios de aceptación de los ítems completados se cumplan efectivamente, y el resultado se registra en Notion.

Para que un requerimiento se considere implementado exitosamente, deben cumplirse simultáneamente las siguientes condiciones: el pull request asociado ha sido aprobado por al menos un integrante distinto al autor, las pruebas automatizadas relevantes pasan en el pipeline de CI, el ítem en GitHub Projects está en estado completado y la Matriz de Trazabilidad ha sido actualizada.

5.2.3. Acciones correctivas

El progreso del proyecto se mide a través de seis métricas principales, coherentes con las definidas en la sección 7.4 del presente documento. A continuación se describe cada una con su unidad de medición, su umbral de aceptación y el responsable de su recolección.

La completitud de tareas se expresa como el porcentaje de tareas del backlog de GitHub Projects marcadas como completadas respecto al total de tareas planificadas para la semana en curso. Una tarea se considera completada únicamente cuando su pull request asociado ha sido aprobado por al menos un integrante distinto al autor y mergeado a la rama de desarrollo. El target semanal es igual o superior al 80%. Esta métrica la recolecta el coordinador cada viernes a las 5:00 p.m., revisando el tablero de GitHub Projects y registrando el porcentaje resultante en Google Spreadsheets bajo la hoja proyecto/metricas-semanales.

La cobertura de pruebas unitarias del núcleo se expresa como el porcentaje de líneas de código del núcleo del SDK cubiertas por pruebas unitarias automatizadas. El umbral mínimo aceptable es del 80%, conforme a lo establecido en el Plan de Aceptación de la sección 3.3.1. Esta métrica la recolecta el líder de arquitectura y núcleo antes de cada merge a la rama de desarrollo, ejecutando jest --coverage en el pipeline de integración continua de GitHub Actions. El resultado se reporta automáticamente en el comentario del pull request correspondiente.

Las métricas de complejidad CK cuantifican las propiedades estructurales del código del

framework mediante tres indicadores del conjunto de Chidamber y Kemerer: WMC (Weighted Methods per Class), con umbral máximo de 15; CBO (Coupling Between Object Classes), con umbral máximo de 5; y RFC (Response For a Class), con umbral máximo de 20. Estas métricas se calculan al cierre de cada iteración de desarrollo ejecutando un script de ts-morph que analiza el directorio src/ y genera un reporte JSON con los valores por clase. Los resultados se registran en Google Spreadsheets bajo la hoja proyecto/metricas-ck. La recolección es responsabilidad del líder de evaluación y documentación. La definición completa de estas métricas y su papel en la evaluación del artefacto se describe en la sección 7.4.

El cumplimiento de hitos es una métrica binaria que indica si cada uno de los seis hitos del ciclo de vida (H1 a H6) fue alcanzado dentro del rango de fecha permitido, definido como la fecha comprometida más o menos tres días. El coordinador verifica esta métrica en la reunión de sincronización de los jueves, contrastando el estado de los entregables asociados a cada hito frente a las fechas del cronograma de la sección 4.

Los defectos por artefacto en revisión contabilizan el número de comentarios clasificados como defecto, es decir, errores funcionales o estructurales y no meras sugerencias, encontrados durante la revisión de pull requests para código o durante la revisión por pares para documentos. El target es un máximo de dos defectos por artefacto antes de su aprobación. El líder de arquitectura y núcleo recolecta esta métrica para los artefactos de código y el líder de evaluación y documentación la recolecta para los artefactos documentales, clasificando cada defecto por tipo: lógico, de estilo o de cobertura.

El tiempo de resolución de impedimentos se expresa en días hábiles transcurridos entre el momento en que un impedimento se registra en Notion o GitHub Issues y el momento en que se marca como resuelto. El target para impedimentos críticos es de máximo dos días hábiles. Esta métrica la recolecta el coordinador semanalmente calculando la diferencia entre las fechas de reporte y resolución de cada impedimento registrado.

El progreso se reporta mediante las siguientes actividades:

La revisión semanal de backlog se realiza cada lunes a las 10:00 a.m. en la reunión de planificación del equipo. Los cuatro integrantes actualizan el estado de sus ítems en GitHub Projects, registran los impedimentos encontrados durante la semana anterior y estiman el trabajo pendiente para la semana que inicia. La duración máxima de esta actividad es de una hora. El coordinador es el responsable de convocarla, de calcular la tasa de completitud de la semana cerrada y de presentar un resumen de las métricas de progreso al equipo.

La sincronización de mitad de semana se realiza cada jueves a las 3:00 p.m. con duración máxima de 45 minutos. El coordinador presenta un resumen de las métricas de la semana en curso, se identifican impedimentos activos y se toman decisiones técnicas urgentes. Si en esta instancia se detecta una desviación significativa en alguna métrica, se convoca una sesión de análisis de causa raíz antes de la reunión del lunes siguiente. El acta queda registrada en Notion con las decisiones y compromisos acordados.

El reporte de progreso al director se entrega quincenalmente los viernes a las 4:00 p.m. mediante reunión en Microsoft Teams o por correo electrónico, según la disponibilidad del director. El reporte incluye el estado de los hitos del ciclo de vida, la tasa de completitud semanal acumulada de la quincena, la cobertura de pruebas del núcleo, los defectos abiertos por componente, los impedimentos activos y su tiempo de resolución, y los valores más recientes de las métricas CK cuando corresponda a la fase en curso. La elaboración del reporte es responsabilidad del coordinador y el director lo utiliza para validar que el progreso es adecuado y para autorizar

cambios de alcance si es necesario.

La demostración al cierre de cada iteración de desarrollo se realiza internamente entre los cuatro integrantes al finalizar cada iteración de la fase de desarrollo (núcleo, adaptadores, API de simulación). En esta instancia se verifica que los criterios de aceptación de los ítems completados se cumplen efectivamente y se registra el resultado en Notion. Un requerimiento se considera implementado exitosamente cuando se cumplen simultáneamente las siguientes condiciones: el pull request asociado fue aprobado por al menos un integrante distinto al autor, las pruebas automatizadas relevantes pasan en el pipeline de CI con cobertura igual o superior al 80%, el ítem en GitHub Projects está en estado completado, los valores de WMC, CBO y RFC de las clases modificadas están dentro de los umbrales definidos, y la Matriz de Trazabilidad ha sido actualizada para reflejar el requerimiento como implementado.

El reporte consolidado de cierre de fase se elabora al alcanzar cada uno de los hitos H1 a H6. El líder de evaluación y documentación prepara un reporte que incluye gráficas de tendencia de las métricas CK a lo largo de la fase, la evolución de la cobertura de pruebas, el cumplimiento de hitos, el número de defectos encontrados y resueltos, y el tiempo promedio de resolución de impedimentos. Este reporte forma parte del documento de trabajo de grado para análisis retrospectivo y se entrega al director junto con el entregable formal de la fase.

Las acciones correctivas se activan bajo las siguientes condiciones.

Cuando la tasa de completitud al cierre de una semana es inferior al 60% de las tareas planificadas, el coordinador convoca dentro de las 24 horas siguientes una reunión extraordinaria de no más de una hora con todos los integrantes. En esta reunión se identifican las causas del retraso mediante un análisis de causa raíz, y se decide por consenso alguna de las siguientes acciones: redistribuir el trabajo entre integrantes con mayor disponibilidad, reducir el alcance del ítem afectado postergando funcionalidad no crítica a la siguiente iteración, o aumentar el tiempo de dedicación semanal de los integrantes involucrados. La decisión queda registrada en Notion con fecha, responsables y fecha de revisión de su efectividad. Si el retraso se origina en un impedimento técnico que ningún integrante puede resolver en menos de dos días hábiles, se escala al director del proyecto en la reunión quincenal o de forma inmediata si el impedimento bloquea el hito en curso.

Cuando un valor de WMC, CBO o RFC supera su umbral en alguna clase del framework durante la revisión del pull request, el líder de arquitectura y núcleo registra el hallazgo como defecto en el pull request y el código no se aprueba hasta que el autor resuelva la violación. Si la violación persiste en dos iteraciones consecutivas sobre la misma clase, se convoca una sesión técnica de refactorización con los integrantes involucrados en ese componente.

Cuando un hito no se alcanza en la fecha comprometida, el equipo elabora dentro de los dos días hábiles siguientes un plan de recuperación que especifica qué actividades se reordenarán, qué recursos adicionales se asignarán y en cuántas semanas se proyecta recuperar el retraso. Este plan se presenta al director en la reunión quincenal inmediatamente siguiente al incumplimiento del hito. En ningún caso se acuerdan reducciones de alcance que afecten los entregables comprometidos sin la aprobación explícita del director.

Cuando la cobertura de pruebas del núcleo cae por debajo del 80% en cualquier ejecución del pipeline de CI, el pull request asociado se rechaza automáticamente y el autor es responsable de agregar las pruebas faltantes antes de solicitar una nueva revisión. Esta condición no es negociable y no requiere reunión de equipo para activarse.

5.3. Cierre del Proyecto

El propósito de esta sección es describir las acciones necesarias para cerrar formalmente cada fase del proyecto antes de cada entrega, así como las acciones de cierre final al término del semestre.

5.3.1. Cierre de fase

Al finalizar cada fase del ciclo de vida DSR, el equipo ejecuta las siguientes actividades previas a la entrega. Se realiza un post-mortem de la fase, de no más de una hora, en el que se identifican las actividades que funcionaron bien, las que presentaron dificultades y las mejoras que se incorporarán en la fase siguiente. Los resultados quedan registrados en Notion. Se verifica que todos los artefactos comprometidos para esa fase estén completos, actualizados y almacenados en los repositorios correspondientes (GitHub para el código y los documentos técnicos, Google Drive para los entregables académicos). Se elabora el reporte gerencial de la fase, que resume el trabajo realizado, el estado de los hitos, las métricas de progreso y las lecciones aprendidas, y se entrega al director junto con el entregable formal de la fase.

5.3.2. Cierre final del proyecto

Al término del semestre, el equipo ejecuta adicionalmente las siguientes acciones de cierre final: publicación del repositorio público del framework con documentación suficiente para que un desarrollador externo pueda integrar el kit sin haber participado en su desarrollo, entrega del documento de trabajo de grado en su versión final al director y a la facultad según los procedimientos académicos vigentes, y archivo de todos los artefactos del proyecto en los repositorios correspondientes con acceso habilitado para el director y los evaluadores académicos.

6. Entrega del Producto

El Kit Pagos Colombia no tiene un cliente externo en el sentido comercial del término. Los destinatarios de la entrega del producto son dos: el Ph.D. Jaime Andrés Pavlich Mariscal en su rol de director del trabajo de grado, quien recibe los artefactos intermedios y aprueba formalmente los entregables clave a lo largo del semestre; y la Facultad de Ingeniería de la Pontificia Universidad Javeriana, que recibe el producto completo al cierre del semestre como parte de los procedimientos académicos del trabajo de grado. Esta distinción implica que el proceso de entrega tiene dos momentos: entregas parciales al director asociadas a los hitos del ciclo de vida DSR, y una entrega final consolidada que cierra el proyecto.

Las entregas parciales ocurren al alcanzar los hitos H3, H4 y H5 definidos en la sección 3.1.3. En cada uno de estos puntos, el Coordinador del equipo elabora y remite al director un reporte de progreso que incluye el estado de los artefactos completados hasta ese momento, junto con evidencias de que cumplen los criterios del Plan de Aceptación correspondiente (sección 3.3). El director valida el avance y emite retroalimentación antes de que el equipo proceda a la siguiente fase.

La entrega final del producto se concentra en el hito H6 y comprende cinco artefactos, conforme a lo establecido en la sección 1.4. La tabla a continuación detalla cada artefacto, su canal de entrega, el responsable y el momento en que se ejecuta la acción de entrega.

Artefacto	**Canal de entrega**	**Responsable de la entrega**	**Momento**	**Criterio de aceptación**
Prototipo funcional del SDK con interfaz unificada	Publicación del paquete en npm (versión estable 1.0.0)	Líder de arquitectura y núcleo SDK	H6	Sección 3.3.1
API de simulación desplegad	URL pública en Railway o Render, accesible desde internet	Líder de integración	H6	Sección 3.3.2
Documentación centralizada de datos de prueba	Repositorio público en GitHub (directorio /docs/sandbox-data)	Líder de integración	H6	Sección 3.3.3
Informe comparativo de métricas de evaluación (WMC, CBO, RFC)	Capítulo de evaluación dentro del documento final de grado	Líder de evaluación y documentación	H6	Sección 3.3.4
Documento final del trabajo de grado y repositorio público	Entrega al director y a la Facultad por los canales institucionales;	Líder de evaluación y documentación	H6	Sección 3.3.5

	repositorio público en GitHub con documentación TypeDoc			

Tabla 10. Entregables.

Ningún artefacto se considera formalmente entregado si no cumple previamente los criterios de aceptación descritos en la sección 3.3. Es responsabilidad del integrante que lidera cada frente verificar el cumplimiento de dichos criterios antes de ejecutar la acción de entrega. El director aprueba la entrega del documento final y el Coordinador del equipo verifica que el repositorio público esté accesible y documentado con suficiente detalle para que un desarrollador externo pueda integrar el kit sin contactar a los autores, conforme al criterio de aceptación de la sección 3.3.5. Las acciones de archivo y cierre formal del proyecto están descritas en la sección 5.3.2.

7. Procesos de Soporte

Describir los procesos transversales que soportan la ejecución del proyecto Kit Pagos Colombia, permitiendo que todos los integrantes del equipo conozcan cuáles son estos procesos y cuáles son sus responsabilidades específicas en cada uno de ellos. Estos procesos abarcan desde la regulación del trabajo en equipo y la administración de riesgos, hasta el control de cambios, la medición del progreso y las actividades de aseguramiento de calidad.

Los procesos de soporte se organizan en cinco áreas funcionales interrelacionadas: ambiente de trabajo (reglas de convivencia y mecanismos de control), administración de riesgos (identificación, valoración y mitigación), administración de configuración y documentación (control de cambios e ítems de configuración), métricas y medición (recolección y consolidación de datos de desempeño), y control de calidad (verificación, validación y auditorías). Cada área se ejecuta de manera continua a lo largo del proyecto, integrándose en los ciclos de iteración DSR definidos en la sección 3.1.

7.1. Ambiente de Trabajo

El equipo Kit Pagos Colombia trabaja bajo un conjunto explícito de reglas diseñadas para asegurar que la comunicación sea clara, que el trabajo esté siempre documentado y que las responsabilidades sean bien distribuidas. El equipo se reúne dos veces por semana de manera obligatoria: una reunión de planificación todos los lunes a las 10:00 a.m. (máximo una hora) donde se define el trabajo para la semana, y una reunión de sincronización los jueves a las 3:00 p.m. (máximo 45 minutos) donde se revisa el avance, se identifican impedimentos y se toman decisiones técnicas. Las actas de todas las reuniones quedan registradas en Notion, incluyendo decisiones, compromisos y cambios de alcance. Fuera de estas reuniones, la comunicación se realiza mediante Discord en el canal #kit-pagos-colombia, que sirve como histórico de conversaciones técnicas y administrativas. Se espera que todos los integrantes respondan consultas dentro de máximo 24 horas hábiles.

Las tareas se asignan al inicio de cada ciclo de dos semanas en la reunión de planificación del lunes. El Coordinador del equipo elabora un listado de tareas esperadas indicando responsable principal y roles de soporte de otros integrantes, considerando el plan de hitos de la sección 3.1.3 y la disponibilidad del equipo. Ningún integrante puede asumir simultáneamente más de 40 horas de trabajo semanal en el proyecto. Todo código debe cumplir con los estándares TypeScript definidos en el documento técnico del SDK (sección 1.4). Antes de que código sea fusionado en la rama principal de GitHub, el Líder arquitectura y núcleo SDK revisa que se hayan ejecutado pruebas unitarias con cobertura mínima del 80%. Todo documento (excepto borradores privados) debe ser revisado por al menos un compañero antes de su entrega final; la revisión se registra mediante comentarios en Google Docs o GitHub. Los artefactos que no cumplan con estos criterios de calidad son devueltos para corrección sin excepción.

La puntualidad en las entregas es crítica. Las entregas de artefactos se realizan en las fechas comprometidas al inicio de cada ciclo. Si un integrante prevé que no podrá cumplir una entrega dentro de 48 horas, notifica de inmediato al Coordinador del equipo y al director del proyecto, explicando la causa y proponiendo una nueva fecha de entrega. Las decisiones sobre extensión de plazo las toma el director del proyecto. El incumplimiento de compromisos sin notificación previa se registra en el acta de reunión y es tema de discusión explícita en la próxima sesión de equipo. En caso de desacuerdo técnico entre dos o más integrantes sobre arquitectura, diseño o decisiones de implementación, se documenta la posición de cada uno en una nota de Notion y se convoca al Líder arquitectura y núcleo SDK o al Coordinador para arbitrar y tomar una decisión consensuada. Si la discrepancia persiste, se escala al director del proyecto. Se preserva en todo momento el respeto y la profesionalidad; los conflictos técnicos nunca deben afectar las relaciones personales del equipo.

Para asegurar que los compromisos se cumplen, cada viernes al cierre de la jornada, el Coordinador del equipo elabora un reporte de completitud de tareas de la semana. Calcula el porcentaje de tareas completadas respecto a tareas planificadas. Si la tasa de completitud es inferior al 60%, se activa

inmediatamente el protocolo de acciones correctivas descrito en la sección 5.2, que incluye una reunión extraordinaria de no más de una hora para identificar causas del retraso y redistribuir o reducir alcance. El reporte de completitud se comparte en el canal #kit-pagos-colombia y se discute detalladamente en la reunión del lunes siguiente. Todas las decisiones arquitectónicas, cambios de alcance y compromisos nuevos se registran en Notion con etiqueta de fecha y responsable, permitiendo que cualquier persona, incluso el director, pueda auditar la trazabilidad histórica de decisiones. Al cierre de cada fase DSR, el Coordinador elabora un sumario de todas las decisiones tomadas durante esa fase, que se incluye en el reporte gerencial de la fase.

7.2. Análisis y Administración de Riesgos

El proyecto enfrenta diversos tipos de riesgos: técnicos (dificultades en integración con pasarelas de pago, limitaciones de APIs, problemas de rendimiento del SDK), organizacionales (cambios en disponibilidad de integrantes, conflictos de prioridades con otros proyectos académicos) y académicos (cambios en criterios de evaluación, retrasos en retroalimentación del director). Para administrar estos riesgos de manera sistemática, el equipo ejecuta un proceso continuo de identificación, valoración y mitigación a lo largo de todo el proyecto.

En la reunión de planificación de cada ciclo de dos semanas, el equipo participa en una sesión de lluvia de ideas (brainstorming) de 30 minutos para identificar nuevos riesgos o cambios en riesgos ya existentes. Se consideran riesgos de todas las categorías mencionadas. El Coordinador documenta todos los riesgos propuestos en Notion sin aplicar ningún filtro inicial; en esta fase no hay criterio, solo se recopilan todas las preocupaciones del equipo. Para cada riesgo identificado, el equipo estima dos variables: probabilidad de ocurrencia (baja: <20%, media: 20-60%, alta: >60%) e impacto esperado en caso de que el riesgo se concrete (bajo: retraso <3 días, medio: retraso 3-7 días, alto: retraso >7 días o cambio de alcance mayor). La prioridad del riesgo se calcula multiplicando probabilidad × impacto, y se clasifican riesgos en tres categorías: verde (prioridad <0.2, monitoreo pasivo), amarillo (0.2-0.6, requiere plan de acción), y rojo (>0.6, requiere escalada inmediata al director). Este análisis completo se registra en el Anexo B del proyecto.

En cada reunión de sincronización (jueves), se revisan los riesgos de categoría amarillo y rojo, actualizando su estado y verificando que las acciones de mitigación se estén ejecutando según lo planeado. La Líder evaluación y documentación prepara un sumario quincenal de cambios en riesgos (nuevos identificados, riesgos que han desaparecido, cambios de prioridad) que se comparte con el director del proyecto. Riesgos nuevos pueden ser propuestos en cualquier momento por cualquier integrante; se añaden inmediatamente a Notion y se evalúan en la próxima reunión de planificación. Para cada riesgo de categoría rojo o amarillo, el equipo define acciones específicas de prevención (actividades para que el riesgo no ocurra) y mitigación (cómo responder si el riesgo sí ocurre). Por ejemplo, para el riesgo técnico de "dificultad en integración con pasarelas", la prevención es realizar pruebas exploratorias tempranas con APIs de sandbox, y la mitigación es diseñar adaptadores específicos si se identifica incompatibilidad. Para el riesgo organizacional de "cambio de disponibilidad de integrantes", la prevención es mantener documentación exhaustiva de decisiones técnicas en GitHub y Google Drive, y la mitigación es asignar responsables alternos para funciones críticas. Cada acción de mitigación tiene un responsable asignado, un plazo de inicio y una descripción de recursos necesarios.

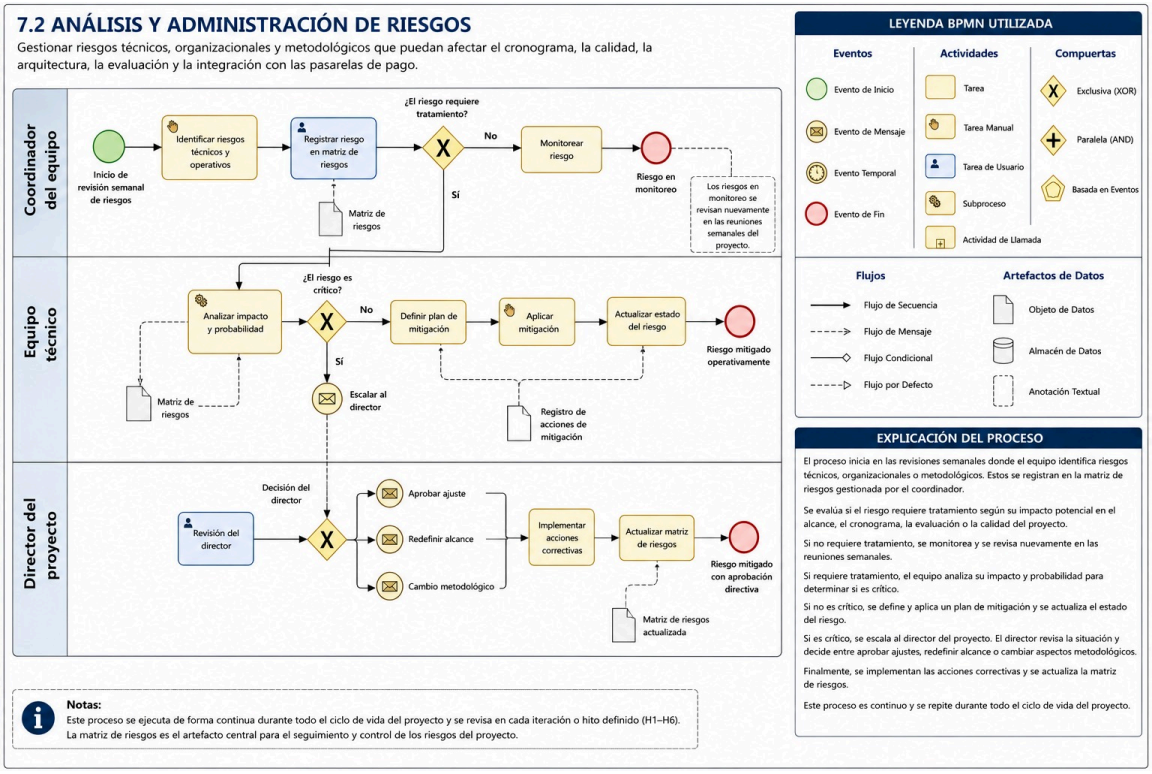


Figura 5. Diagrama BPMN para administración de requerimientos

7.3. Administración de Configuración y Documentación

El proyecto produce artefactos de varios tipos, código fuente, código de pruebas, documentación técnica, documentación académica y datos de configuración, que evolucionan a lo largo de los ciclos DSR. Para mantener trazabilidad, consistencia y la capacidad de retroceder a versiones anteriores si es necesario, se implementa un proceso riguroso de control de cambios.

Los ítems de configuración principales son identificados explícitamente. El código fuente del SDK en TypeScript (directorio src/) y el código de la API de simulación Fastify (directorio api/) se versionan en GitHub utilizando versionamiento semántico (MAJOR.MINOR.PATCH). Las pruebas unitarias (directorio test/) se mantienen sincronizadas con cambios en src/. La rama principal es main (código estable listo para publicación), la rama de desarrollo es dev (integración de nuevas features), y cada feature individual tiene su propia rama feature/nombre-descriptivo que se fusiona en dev una vez completada. La documentación técnica se almacena en docs/ en formato Markdown: architecture.md (descripción de patrones y componentes), sandbox-data.md (especificación de escenarios de prueba por pasarela), integration-guide.md (tutorial para desarrollador externo), y CHANGELOG.md (historial de cambios por versión). La documentación de APIs se genera automáticamente mediante TypeDoc cada build. La documentación académica (documento de trabajo de grado, reportes de fase) se almacena en Google Drive con historial de cambios y versionamiento manual (v0.1, v0.2, etc.). Los datos de configuración se mantienen en Google Sheets: plan de riesgos (Anexo B), plan de mitigación (Anexo C), y matriz de trazabilidad (sección 1.3).

Para cambios en código, se sigue un proceso estructurado: el desarrollador crea una rama feature/nombre-descriptivo, implementa el cambio localmente, ejecuta pruebas unitarias (jest --coverage) asegurando cobertura mínima del 80%, pushea el código a GitHub, y crea una Pull Request (PR) describiendo brevemente el cambio. El CI/CD pipeline automático (GitHub Actions) compila el código, ejecuta pruebas, verifica coverage y ejecuta análisis de estilo (ESLint). Si hay errores, el desarrollador los corrige localmente e itera. Una vez que el build es exitoso, el Líder arquitectura y núcleo SDK y al menos otro integrante revisan el código verificando lógica correcta, aplicación de patrones de diseño GoF, y métricas de complejidad (WMC ≤15, CBO ≤5, RFC ≤20). Si la revisión identifica mejoras necesarias, se registran como comentarios en el PR; el desarrollador las aborda e itera. Una vez aprobado, se fusiona (merge) en la rama dev. Para cambios en documentación, el proceso es similar pero sin rama de feature: el autor modifica el documento en GitHub (si es .md) o Google Docs, otro integrante revisa verificando gramática, claridad y validez de referencias cruzadas, y una vez aprobado se considera finalizado. Al cierre de cada fase DSR (fin de H1, H2, etc.), se consolida la rama dev en main, se genera un tag de versión (ej. v0.1.0, v0.2.0), se publica la versión estable en npm, y se actualiza CHANGELOG.md con descripción de cambios incluidos.

La tabla de evolución de artefactos (Tabla en sección 7) define cuándo cada artefacto se crea, se refina, se expande, se optimiza, se publica o se archiva durante los hitos H1 a H6. Por ejemplo, el SDK src/ se crea en H1, se refina en H2, se expande en H3, se optimiza en H4, se finaliza en H5 y se publica en H6. La API de simulación se crea en H2, se expande en H3-H4, se despliega en H5 y se finaliza en H6. La documentación del trabajo de grado se comienza en H1 con un outline, se elaboran capítulos en cada fase posterior, y se entrega versión final en H6. Este cronograma asegura que todos los artefactos estén listos en el momento correcto y evolucionen gradualmente a lo largo del proyecto.

Artefacto	**Ubicación**	**Tipo**	**Ciclo de vida (Hitos H1-H6)**
SDK TypeScript /src	GitHub (main/dev)	Código fuente	H1: Crear H2: Refinar H3: Expandir H4: Optimizar H5: Finalizar H6: Publicar
Pruebas unitarias /test	GitHub (main/dev)	Código de pruebas	H1: Crear H2-H4: Expandir H5: Mantener H6: Auditar
API Fastify /api	GitHub (main/dev	Código fuente	H2: Crear H3-H4: Expandir H5: Desplegar H6: Finalizar
Documento de arquitectura	GitHub	Documentación técnica	H1: Crear H2-H5: Refinar

/docs/architecture.md			H6: Publicar
Datos de prueba (sandbox) /docs/sandbox-data.md	GitHub	Documentación técnica	H2: Crear H3-H4: Expandir H5: Consolidar H6: Publicar
Guía de integración /docs/integration-guide .md	GitHub	Documentación técnica	H3: Crear H4-H5: Refinar H6: Publicar
Registro de cambios /CHANGELOG.md	GitHub	Documentación técnica	H1: Crear H2-H5: Actualizar H6: Consolidar
Documentación API (TypeDoc) Generado automático	GitHub Pages	Documentación técnica	H1: Crear H2-H5: Regenerar H6: Publicar
Trabajo de grado Documento final	Google Drive	Documentación académica	H1: Esbozo H2-H4: Elaborar H5: Revisar H6: Entregar
Reportes de fase Documento	Google Drive	Documentación académica	H1,H2,H3: Crear H4,H5: Crear H6: Crear
Matriz de trazabilidad Google Sheets	Google Drive	Datos de configuración	H1: Crear H2-H5: Actualizar H6: Consolidar
Plan de riesgos Anexo B	Google Sheets	Datos de configuración	H1: Crear H2-H5: Mantener H6: Archivar
Plan de mitigación Anexo C	Google Sheets	Datos de configuración	H1: Crear H2-H5: Mantener H6: Archivar
Métricas de complejidad (CK) Google Sheets	Google Drive	Datos de configuración	H2: Crear H3-H5: Recolectar H6: Reportar

Tabla 11. Descripción de artefactos de documentación.

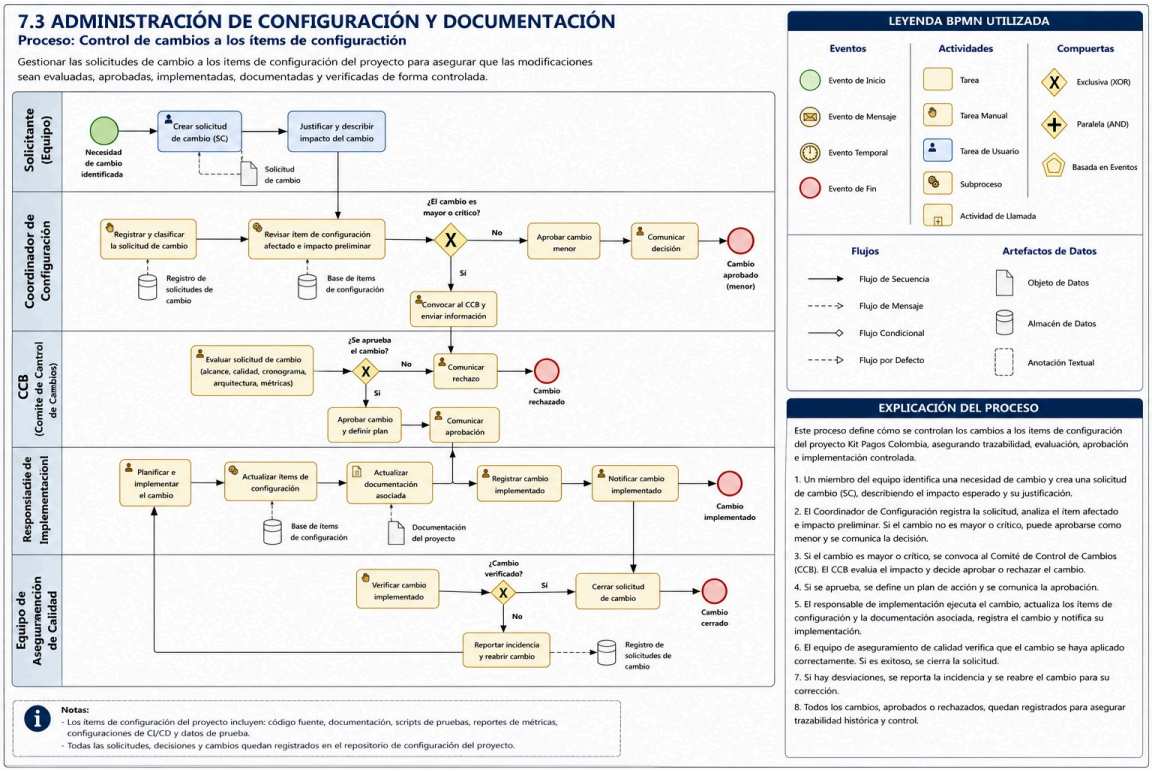


Figura 6. Diagrama BPMN para documentación

7.4. Métricas y Proceso de Medición

Para que el proyecto pueda ser gestionado efectivamente, es esencial medir el progreso, la calidad de los artefactos y la eficiencia del equipo. Se definen seis métricas principales: completitud de tareas (porcentaje de tareas completadas vs. planificadas cada semana, target ≥80%), cobertura de pruebas unitarias (porcentaje de líneas de código ejecutadas por tests en el SDK, target ≥80%), métricas de complejidad CK (WMC ≤15, CBO ≤5, RFC ≤20 por clase), cumplimiento de hitos (número de hitos completados en fecha ±3 días), defectos en revisión (número de defectos por artefacto, target ≤2 por artefacto), y tiempo de resolución de impedimentos (días hábiles entre reporte y resolución, target ≤2 días para impedimentos críticos).

La recolección de métricas se realiza de manera específica para cada métrica. La completitud de tareas la recolecta el Coordinador cada viernes a las 5:00 p.m., revisando el board Notion del ciclo actual, contando tareas completadas vs. planificadas, y registrando el porcentaje en Google Sheets (proyecto/metricas-semanales). La cobertura de pruebas la recolecta el Líder SDK antes de cada merge a dev, ejecutando jest --coverage en el pipeline de CI/CD de GitHub Actions; el resultado se reporta automáticamente en el comentario del PR. Las métricas CK las recolecta el Líder SDK al final de cada ciclo de dos semanas, ejecutando un script de ts-morph que analiza el directorio src/ y genera un reporte JSON de WMC, CBO, RFC por clase; los valores se registran en Google Sheets (proyecto/metricas-ck). El cumplimiento de hitos lo recolecta el Coordinador cada jueves en la reunión de sincronización, revisando si los entregables asociados a cada hito están completos dentro del rango de fecha permitido (±3 días). Los defectos en revisión los recolectan el Líder SDK (para código) y el Líder Documentación (para documentos), contando comentarios de tipo "defecto" (no sugerencia, sino error) en PRs o Google Docs, y clasificándolos por tipo (lógico, estilo, cobertura). El tiempo de resolución de impedimentos lo recolecta el Coordinador, registrando en Notion o GitHub la fecha de reporte y fecha de resolución de cada impedimento, y calculando la diferencia en días hábiles.

La información consolidada de métricas se utiliza de tres maneras. En las reuniones de sincronización semanales (jueves), el Coordinador presenta un resumen de métricas de la semana anterior; si hay desviaciones significativas (ej. completitud <60%), se convoca una sesión de análisis de causa raíz. En los reportes de cierre de fase (al cierre de cada H1-H6), el Líder Documentación prepara un reporte consolidado de métricas del ciclo completado, incluyendo gráficas de tendencia de complejidad (WMC/CBO/RFC), cobertura de pruebas, cumplimiento de hitos y defectos encontrados; este reporte forma parte del documento de trabajo de grado para análisis retrospectivo. En la reunión quincenal con el director (viernes 4:00 p.m.), el Coordinador comparte un resumen ejecutivo de métricas clave (cumplimiento de hitos, estado de riesgos, impedimentos abiertos), que el director utiliza para validar que el progreso es adecuado y para autorizar cambios de alcance si es necesario.

En resumen, las metricas que se usaran son las siguientes:

- **Completitud de tareas**

La métrica de completitud de tareas mide el porcentaje de tareas completadas frente a las tareas planificadas en cada periodo de seguimiento. Esta métrica se relaciona principalmente con las secciones 5.2.1, 5.2.3 y 7.4 del SPMP. Se encuentra asociada a los planes de control de cambios, monitoreo y gestión de recursos. La meta establecida es mantener un cumplimiento igual o superior al 80%. Su seguimiento se realizará semanalmente cada viernes a las 5:00 p.m., y el responsable de

su monitoreo será el coordinador del proyecto.

- **Cobertura de pruebas unitarias**

La cobertura de pruebas unitarias mide el porcentaje de líneas de código ejecutadas mediante pruebas automatizadas sobre el SDK y la API de simulación. Esta métrica toma como referencia principal las secciones 3.3.1, 5.2.3 y 7.4. Está asociada a los planes de aceptación del producto, control de cambios y aseguramiento de calidad. La meta definida es alcanzar una cobertura igual o superior al 80%, criterio considerado no negociable para la aceptación del producto. La medición se ejecutará automáticamente en cada pull request mediante integración continua, y el responsable será el líder del SDK.

- **WMC (Weighted Methods per Class)**

La métrica WMC evalúa la complejidad de los métodos implementados dentro de las clases principales del framework. Sus referencias principales dentro del documento corresponden a las secciones 3.3.4, 5.2.3 y 7.4. Esta métrica se relaciona con los planes de aceptación del producto, evaluación DSR y cierre de fase. La meta establecida es mantener un valor menor o igual a 15. Su revisión se realizará cada dos semanas y al finalizar cada iteración del desarrollo, bajo la responsabilidad del líder del SDK.

- **CBO (Coupling Between Object Classes)**

La métrica CBO mide el nivel de acoplamiento entre clases del sistema, permitiendo evaluar la dependencia estructural entre componentes del framework. Esta métrica referencia las secciones 3.3.4, 5.2.3 y 7.4 del SPMP. Se encuentra asociada a los planes de aceptación del producto, evaluación DSR y cierre de fase. La meta definida es mantener un valor menor o igual a 5. La medición se efectuará cada dos semanas y al cierre de cada iteración, siendo responsable el líder del SDK.

- **RFC (Response For a Class)**

La métrica RFC mide la cantidad potencial de métodos que pueden ejecutarse como respuesta a una solicitud realizada sobre una clase. Sus referencias principales son las secciones 3.3.4, 5.2.3 y 7.4. Está asociada a los planes de aceptación del producto, evaluación DSR y cierre de fase. La meta establecida es mantener un valor menor o igual a 20. Esta métrica será revisada cada dos semanas y al finalizar cada iteración del proyecto, bajo responsabilidad del líder del SDK.

- **Cumplimiento de hitos**

La métrica de cumplimiento de hitos verifica que los hitos H1 a H6 definidos en el ciclo de vida del proyecto se completen dentro del margen permitido de ±3 días respecto al cronograma planificado. Las referencias principales de esta métrica corresponden a las secciones 3.1.3, 5.2.3 y 7.4. Se relaciona con los planes de ciclo de vida, monitoreo y acción correctiva. La meta establecida es lograr un cumplimiento del 100% de los hitos definidos. Su seguimiento será semanal cada jueves a las 3:00 p.m., y el responsable será el coordinador del proyecto.

- **Defectos encontrados en revisión**

Esta métrica contabiliza la cantidad de errores detectados durante las revisiones de código y documentación antes de integrar cambios a la rama principal del repositorio. Las referencias principales corresponden a las secciones 5.2.3, 7.3 y 7.5. Se asocia con los planes de control de cambios, calidad y cierre de fase. La meta establecida es mantener un máximo de dos defectos por artefacto revisado. La medición se realizará por cada revisión de código o documento efectuada, y los responsables serán los líderes técnicos del proyecto.

- **Tiempo de resolución de impedimentos críticos**

Esta métrica mide el tiempo requerido para resolver bloqueos críticos que afecten el desarrollo normal del proyecto, incluyendo problemas de infraestructura, integración o dependencias externas. Sus referencias principales son las secciones 5.2.3 y 7.4. Está asociada a los planes de monitoreo, reportes y acción correctiva. La meta establecida es resolver cada impedimento crítico en un tiempo máximo de dos días hábiles. El seguimiento se realizará semanalmente cada jueves, y el responsable será el coordinador del proyecto.

7.5. Control de Calidad

El proyecto Kit Pagos Colombia implementa un modelo de control de calidad con múltiples capas, asegurando que código, documentación y datos de prueba cumplan con estándares definidos antes de su entrega. Los procesos de QA abarcan pruebas unitarias (antes de integración), revisión de código en PR (verificación de arquitectura y complejidad), análisis de complejidad CK (al cierre de ciclo), revisión de documentación (verificación de gramática y coherencia), validación de datos de prueba (ejecución de escenarios contra APIs), pruebas de integración manual (verificación fin a fin), validación de entrega final contra Plan de Aceptación, y auditoría de configuración (verificación de que repositorios y documentación están públicos y correctamente organizados).

Cada tipo de artefacto tiene criterios explícitos de aceptación. Para código TypeScript: debe compilar sin errores (npm run build), todas las pruebas unitarias deben pasar (npm test), cobertura debe ser ≥80%, debe cumplir estándares ESLint, y métricas de complejidad deben respetar WMC ≤15, CBO ≤5, RFC ≤20. El código debe ser revisado y aprobado por el Líder SDK y al menos otro integrante antes de fusionarse en dev. Para la API REST: debe iniciar sin errores, debe responder correctamente a los 16 escenarios de prueba (4 escenarios × 4 pasarelas: Wompi, PayU, Mercado Pago, Kushki), debe tener documentación de endpoints en OpenAPI/Swagger, debe desplegarse exitosamente en Railway o Render, y debe ser accesible públicamente en una URL estable. Para documentación técnica en Markdown: debe estar libre de errores de ortografía y gramática, debe incluir ejemplos de código o pseudocódigo, debe tener referencias cruzadas válidas, debe tener imágenes/diagramas con etiquetas descriptivas, y debe ser revisada y aprobada por el Líder Documentación y otro integrante. Para el documento de trabajo de grado: debe cumplir con el formato requerido por la Pontificia Universidad Javeriana (márgenes, tipografía, referencias), deben estar completas todas las secciones 1-9, debe tener coherencia y fluidez narrativa entre capítulos, todas las tablas y figuras deben estar numeradas con captions descriptivos, la bibliografía debe estar en formato APA, y debe ser revisado por el director del proyecto y evaluadores académicos designados.

Las responsabilidades en QA están claramente asignadas. El desarrollador/autor es responsable de escribir código y documentación que cumpla con estándares, incluir pruebas, compilar correctamente, y responder comentarios de revisión dentro de 24 horas. El Líder SDK revisa todo código, verifica arquitectura y complejidad, autoriza merge, y mantiene estándares de cobertura. El Líder Integración diseña y valida datos de prueba, ejecuta tests manuales, y documenta resultados. El Líder Documentación revisa documentación, verifica gramática y claridad, y mantiene referencias. El Coordinador registra todas las revisiones, consolida métricas de QA, reporta tendencias, y escala defectos críticos. El director revisa el documento final y autoriza entrega.

Cabe resaltar que todos estos procesos de soporte, ambiente de trabajo, administración de riesgos, control de cambios, medición y QA, no funcionan de manera aislada, sino que están integrados en el ciclo de vida DSR del proyecto (sección 3). Los riesgos se monitorean y actualizan en cada ciclo; las métricas se recolectan continuamente y se utilizan para informar decisiones; los artefactos evolucionan según la tabla de hitos; y la calidad se asegura mediante revisiones realizadas justo antes de cada entrega planificada. Este enfoque holístico de procesos de soporte proporciona al equipo la estructura,

visibilidad y mecanismos de control necesarios para ejecutar el proyecto de manera profesional y entregar resultados de calidad.

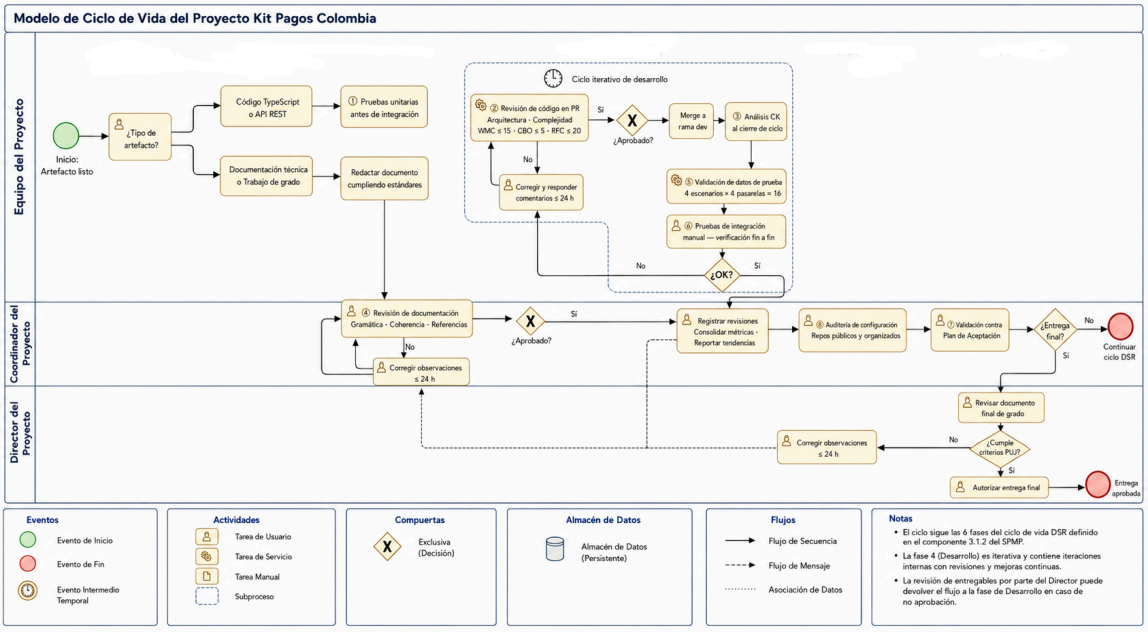


Figura 7. Diagrama BPMN del proceso de control de calidad y validación de entregables

8. Referencias

Ammann, P., & Offutt, J. (2016). Introduction to software testing. Cambridge University Press. https://doi.org/10.1017/9781316771273

Amazon Web Services. (s. f.). ¿Qué es un marco en programación e ingeniería? https://aws.amazon.com/es/what-is/framework/

Qué es el SDK (paquete de desarrollo de software)—AWS. (s. f.). Amazon Web Services, Inc. Recuperado 9 de mayo de 2026, de https://aws.amazon.com/es/what-is/sdk/

Cockburn, A. (2024). Hexagonal architecture explained.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Tesis doctoral, University of California, Irvine).

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Hisyam, M., & Manuaba, I. B. K. (2022). Integration model of multiple payment gateways for online split payment scenario. En 2022 International Conference on Information Management and Technology (ICIMTech) (pp. 122–126). https://doi.org/10.1109/ICIMTech55957.2022.9915168

Hohpe, G., & Woolf, B. (2003). Enterprise integration patterns: Designing, building, and deploying messaging solutions. Addison-Wesley.

Informe de comercio minorista / diciembre 2025. (s. f.). Cámara Colombiana de Comercio Electrónico. Recuperado el 16 de marzo de 2026, de https://ccce.org.co/noticias/informe-de-comercio-minorista/

IEEE. (1998). IEEE recommended practice for software requirements specifications (IEEE Std 830-1998). https://doi.org/10.1109/IEEESTD.1998.88286

Peffers, K., Tuunanen, T., Rothenberger, M., & Chatterjee, S. (2007). A design science research methodology for information systems research. Journal of Management Information Systems, 24(3), 45–77. https://www.researchgate.net/publication/284503626_A_design_science_research_methodology_for_information_systems_research

Taherkordi, A., Romero, D., Rouvoy, R., & Eliassen, F. (2011). REST: From research to practice.

Chidamber, S. R., & Kemerer, C. F. (1994). A metrics suite for object-oriented design. IEEE Transactions on Software Engineering, 20(6), 476–493. https://doi.org/10.1109/32.295895

Hexagonal architecture pattern—AWS Prescriptive Guidance. (s. f.). Amazon Web Services. Recuperado el 15 de abril de 2026, de

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/hexagonal-architecture.html

ISO/IEC/IEEE International Standard—Systems and software engineering–Content management for product life cycle, user and service management information for users. (2023). Recuperado el 5 de Mayo de 2026 ISO/IEC/IEEE 26531:2023(E), 1-68. https://doi.org/10.1109/IEEESTD.2023.10085732

Apache License, Version 2.0 | Apache Software Foundation. (s. f.). Recuperado 9 de mayo de 2026, de https://www.apache.org/licenses/LICENSE-2.0

Patrones de diseño GoF por página. (s. f.). Scribd. Recuperado el 15 de abril de 2026, de https://es.scribd.com/document/388374728/Patrones-de-Diseno-GoF-Por-Pagina